

```

    }

    UserListView.ItemsSource = AccountHelper.AccountList;
    UserListView.SelectionChanged += UserSelectionChanged;
}

/// <summary>
/// Function called when an account is selected in the list of
accounts
/// Navigates to the Login page and passes the chosen account
/// </summary>
private void UserSelectionChanged(object sender,
RoutedEventArgs e)
{
    if (((ListView)sender).SelectedItem != null)
    {
        Account account = (Account)
((ListView)sender).SelectedItem;
        if (account != null)
        {
            Debug.WriteLine($"Account {account.Username}
selected!");
        }
        Frame.Navigate(typeof(Login), account);
    }
}

/// <summary>
/// Function called when the "+" button is clicked to add a new
user.
/// Navigates to the Login page with nothing filled out
/// </summary>
private void AddUserButton_Click(object sender, RoutedEventArgs
e)
{
    Frame.Navigate(typeof(Login));
}
}
}

```

- There are a few places in the app where you want to navigate to the **UserSelection** page. In **MainPage.xaml.cs**, you should navigate to the **UserSelection** page instead of the **Login** page. While you are in the loaded event in **MainPage**, you will need to load the accounts list so the **UserSelection** page can check if there are any accounts. This will require changing the `Loaded` method to be async and also adding a using statement for the `WindowsHelloLogin.Utils` namespace.

```

using WindowsHelloLogin.Utils;

private async void MainPage_Loaded(object sender, RoutedEventArgs e)
{
    // Load the local account list before navigating to the
    UserSelection page
    await AccountHelper.LoadAccountListAsync();
    Frame.Navigate(typeof(UserSelection));
}

```

- Next, the app will need to navigate to the **UserSelection** page from the **Welcome** page. In both `Click` events, you should navigate back to the **UserSelection** page.

CS

```

private void Button_Restart_Click(object sender, RoutedEventArgs e)
{
    Frame.Navigate(typeof(UserSelection));
}

private void Button_Forget_User_Click(object sender, RoutedEventArgs e)
{
    // Remove it from Windows Hello
    WindowsHelloHelper.RemoveWindowsHelloAccountAsync(_activeAccount);

    // Remove it from the local accounts list and re-save the updated
    list
    AccountHelper.RemoveAccount(_activeAccount);

    Debug.WriteLine($"User {_activeAccount.Username} deleted.");

    // Navigate back to UserSelection page.
    Frame.Navigate(typeof(UserSelection));
}

```

- In the **Login** page, you need code to log in to the account selected from the list in the **UserSelection** page. In the `OnNavigatedTo` event, store the account passed during navigation. Start by adding a new private variable that will identify if the account is an existing account. Then handle the `OnNavigatedTo` event.

CS

```

namespace WindowsHelloLogin.Views
{
    public sealed partial class Login : Page
    {
        private Account _account;
        private bool _isExistingAccount;
    }
}

```

```

public Login()
{
    InitializeComponent();
}

/// <summary>
/// Function called when this frame is navigated to.
/// Checks to see if Windows Hello is available and if an
account was passed in.
/// If an account was passed in set the "_isExistingAccount"
flag to true and set the _account.
/// </summary>
protected override async void OnNavigatedTo(NavigationEventArgs
e)
{
    // Check Windows Hello is set up and available on this
machine
    if (await
WindowsHelloHelper.WindowsHelloAvailableCheckAsync())
    {
        if (e.Parameter != null)
        {
            _isExistingAccount = true;
            // Set the account to the existing account being
passed in
            _account = (Account)e.Parameter;
            UsernameTextBox.Text = _account.Username;
            await SignInWindowsHelloAsync();
        }
    }
    else
    {
        // Windows Hello is not set up, so inform the user
        WindowsHelloStatus.Background = new
SolidColorBrush(Windows.UI.Color.FromArgb(255, 50, 170, 207));
        WindowsHelloStatusText.Text = $"Windows Hello is not
set up!{Environment.NewLine}Please go to Windows Settings and set up a
PIN to use it.";
        LoginButton.IsEnabled = false;
    }
}
}

```

- The **SignInWindowsHelloAsync** method will need to be updated to sign in to the selected account. The **WindowsHelloHelper** will need another method to open the account with Windows Hello, as the account already has an account key created for it. Implement the new method in **WindowsHelloHelper.cs** to sign in an existing user with Windows Hello. For information on each part of the code, please read through the code comments.

```

/// <summary>
/// Attempts to sign a message using the account key on the system for
the accountId passed.
/// </summary>
/// <returns>Boolean representing if creating the Windows Hello
authentication message succeeded</returns>
public static async Task<bool>
GetWindowsHelloAuthenticationMessageAsync(Account account)
{
    KeyCredentialRetrievalResult openKeyResult = await
KeyCredentialManager.OpenAsync(account.Username);
    // Calling OpenAsync will allow the user access to what is
available in the app and will not require user credentials again.
    // If you wanted to force the user to sign in again you can use the
following:
    // var consentResult = await
Windows.Security.Credentials.UI.UserConsentVerifier.RequestVerification
Async(account.Username);
    // This will ask for the either the password of the currently
signed in Microsoft Account or the PIN used for Windows Hello.

    if (openKeyResult.Status == KeyCredentialStatus.Success)
    {
        // If OpenAsync has succeeded, the next thing to think about is
whether the client application requires access to backend services.
        // If it does here you would request a challenge from the
server. The client would sign this challenge and the server
        // would check the signed challenge. If it is correct, it would
allow the user access to the backend.
        // You would likely make a new method called RequestSignAsync
to handle all this.
        // For example, RequestSignAsync(openKeyResult);
        // Refer to the second Windows Hello sample for information on
how to do this.

        // For this sample, there is not concept of a server
implemented so just return true.
        return true;
    }
    else if (openKeyResult.Status == KeyCredentialStatus.NotFound)
    {
        // If the account is not found at this stage. It could be one
of two errors.
        // 1. Windows Hello has been disabled
        // 2. Windows Hello has been disabled and re-enabled cause the
Windows Hello Key to change.
        // Calling CreateWindowsHelloKeyAsync and passing through the
account will attempt to replace the existing Windows Hello Key for that
account.
        // If the error really is that Windows Hello is disabled then
the CreateWindowsHelloKeyAsync method will output that error.
        if (await CreateWindowsHelloKeyAsync(account.Username))
        {
            // If the Hello Key was again successfully created, Windows

```

```

Hello has just been reset.
        // Now that the Hello Key has been reset for the account
retry sign in.
        return await
GetWindowsHelloAuthenticationMessageAsync(account);
    }
}

// Can't use Windows Hello right now, try again later
return false;
}

```

- Update the **SignInWindowsHelloAsync** method in Login.xaml.cs to handle the existing account. This will use the new method in the WindowsHelloHelper.cs. If successful the account will be signed in and the user navigated to the **Welcome** page.

```

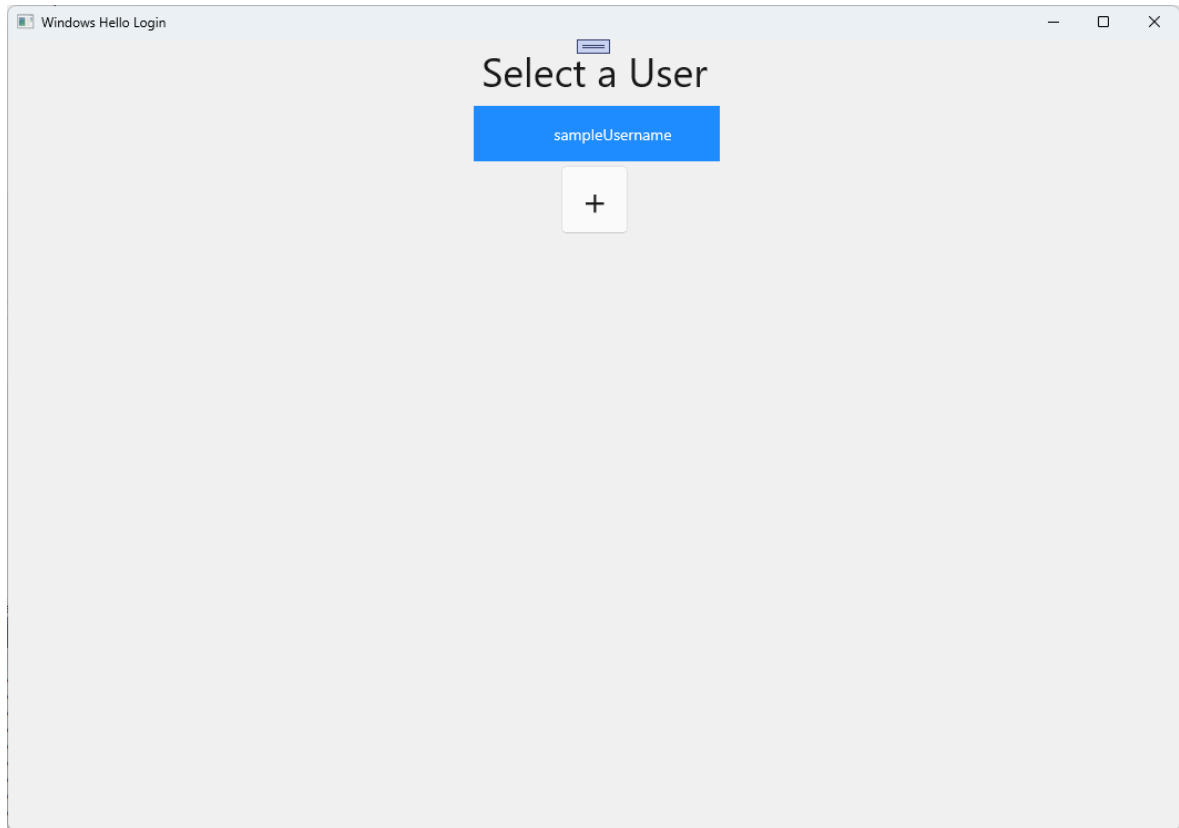
CS

private async Task SignInWindowsHelloAsync()
{
    if (_isExistingAccount)
    {
        if (await
WindowsHelloHelper.GetWindowsHelloAuthenticationMessageAsync(_account))
        {
            Frame.Navigate(typeof(Welcome), _account);
        }
    }
    else if
(AccountHelper.ValidateAccountCredentials(UsernameTextBox.Text))
    {
        //Create and add a new local account
        _account = AccountHelper.AddAccount(UsernameTextBox.Text);
        Debug.WriteLine("Successfully signed in with traditional
credentials and created local account instance!");

        if (await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(UsernameTextBox.Text))
        {
            Debug.WriteLine("Successfully signed in with Windows
Hello!");
            Frame.Navigate(typeof(Welcome), _account);
        }
    }
    else
    {
        ErrorMessage.Text = "Invalid Credentials";
    }
}

```

- Build and run the application. Login with "sampleUsername". Type in your PIN and if successful you will be navigated to the **Welcome** page. Click back to user list. You should now see a user in the list. If you click on this, WindowsHello enables you to sign back in without having to re-enter any passwords etc.



Exercise 3: Registering a new Windows Hello user

In this exercise, you create a new page that can create a new account with Windows Hello. This works similarly to how the **Login** page works. The **Login** page is implemented for an existing user that is migrating to use Windows Hello. A **WindowsHelloRegister** page will create Windows Hello registration for a new user.

- In the **Views** folder, create a new blank page named "WindowsHelloRegister.xaml". In the XAML add in the following to setup the user interface. The interface on this page is similar to the **Login** page.

XML

```
<Grid>
    <StackPanel>
        <TextBlock x:Name="Title" Text="Register New Windows Hello User"
            FontSize="24" Margin="4" TextAlignment="Center"/>

        <TextBlock x:Name="ErrorMessage" Text="" FontSize="20" Margin="4"
```

```

Foreground="Red" TextAlignment="Center"/>

    <TextBlock Text="Enter your new username below" Margin="0,0,0,20"
        TextWrapping="Wrap" Width="300"
        TextAlignment="Center" VerticalAlignment="Center"
FontSize="16"/>

    <TextBox x:Name="UsernameTextBox" Margin="4" Width="250"/>

    <Button x:Name="RegisterButton" Content="Register"
Background="DodgerBlue" Foreground="White"
        Click="RegisterButton_Click_Async" Width="80"
HorizontalAlignment="Center" Margin="0,20"/>

    <Border x:Name="WindowsHelloStatus" Background="#22B14C" Margin="4"
Height="100">
        <TextBlock x:Name="WindowsHelloStatusText" Text="Windows Hello is
ready to use!" FontSize="20"
            Margin="4" TextAlignment="Center"
VerticalAlignment="Center"/>
    </Border>
</StackPanel>
</Grid>

```

- In the WindowsHelloRegister.xaml.cs code-behind file, implement a private `Account` variable and a `Click` event for the register button. This will add a new local account and create a Windows Hello key.

```

CS

using Microsoft.UI.Xaml.Controls;
using Microsoft.UI.Xaml;
using WindowsHelloLogin.Models;
using WindowsHelloLogin.Utills;

namespace WindowsHelloLogin.Views
{
    public sealed partial class WindowsHelloRegister : Page
    {
        private Account _account;

        public WindowsHelloRegister()
        {
            InitializeComponent();
        }

        private async void RegisterButton_Click_Async(object sender,
RoutedEventArgs e)
        {
            ErrorMessage.Text = "";

            // In the real world, you would validate the entered

```

```

credentials and information before
    // allowing a user to register a new account.
    // For this sample, we'll skip that step and just register
an account if the username is not null.

    if (!string.IsNullOrEmpty(UsernameTextBox.Text))
    {
        // Register a new account
        _account =
AccountHelper.AddAccount(UsernameTextBox.Text);
        // Register new account with Windows Hello
        await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(_account.Username);
        // Navigate to the Welcome page.
        Frame.Navigate(typeof(Welcome), _account);
    }
    else
    {
        ErrorMessage.Text = "Please enter a username";
    }
}
}
}

```

- You need to navigate to this page from the **Login** page when register is clicked.

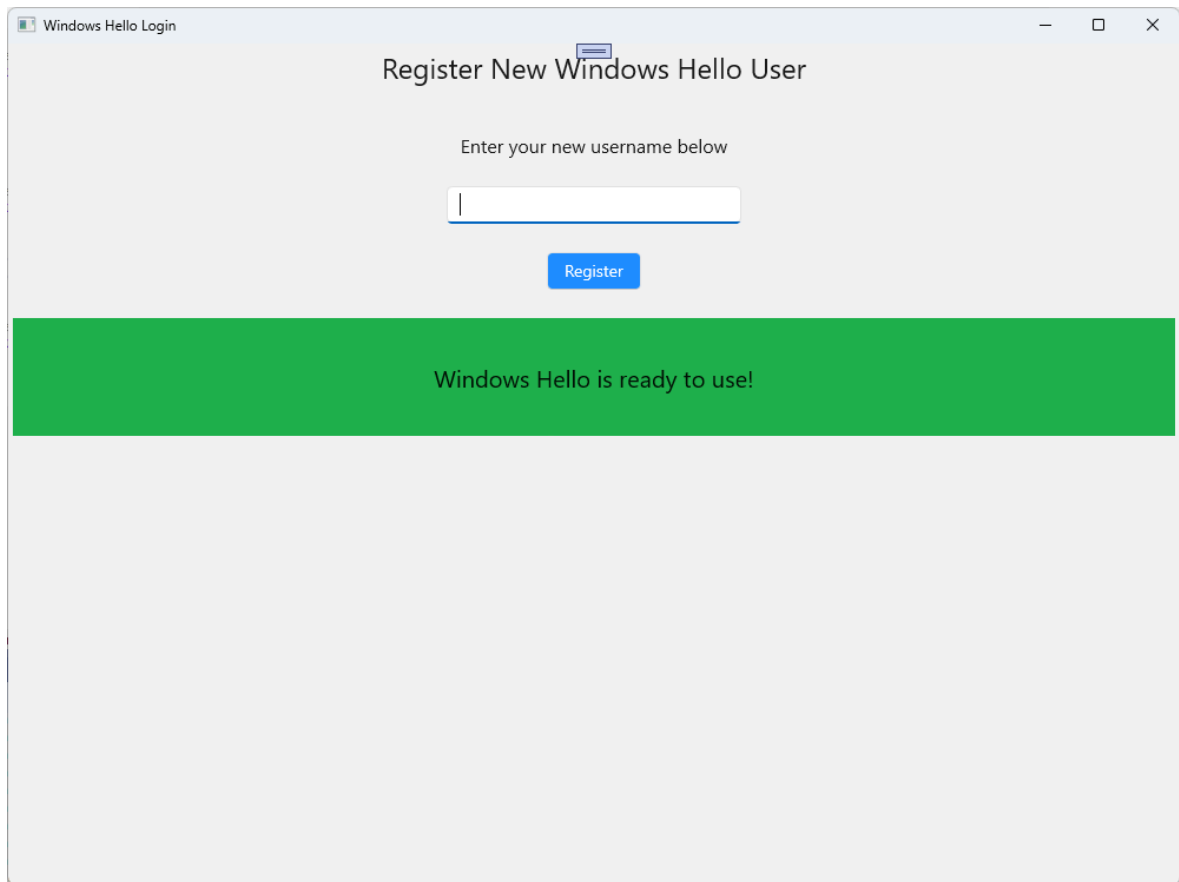
CS

```

private void RegisterButtonTextBlock_OnPointerPressed(object sender,
PointerRoutedEventArgs e)
{
    ErrorMessage.Text = "";
    Frame.Navigate(typeof(WindowsHelloRegister));
}

```

- Build and run the application. Try to register a new user. Then return to the user list and validate that you can select that user and login.



In this lab, you have learned the essential skills needed to use the new Windows Hello API to authenticate existing users and create accounts for new users. With this new knowledge, you can start removing the need for users to remember passwords for your application, yet remain confident that your applications remain protected by user authentication. Windows uses Windows Hello's new authentication technology to support its biometrics login options.

Related topics

- [Windows Hello](#)
- [Windows Hello login service](#)

Create a Windows Hello login service

Article • 07/11/2024

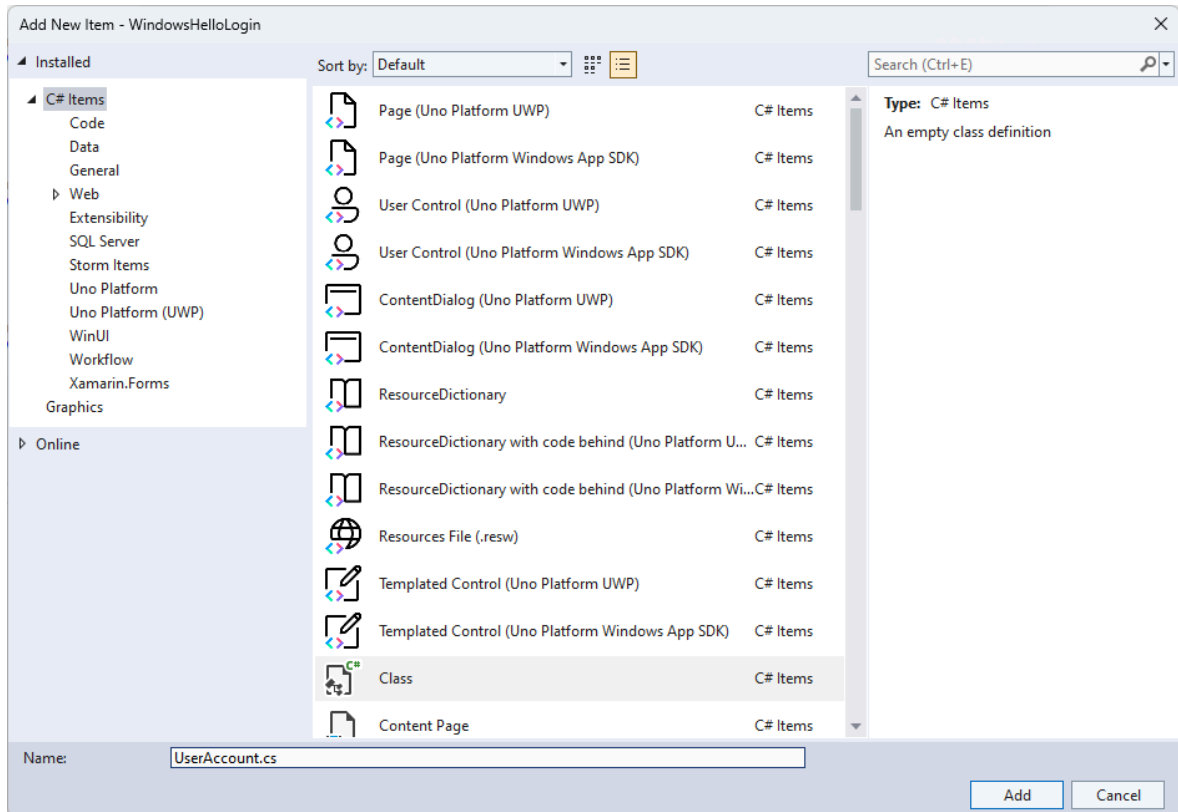
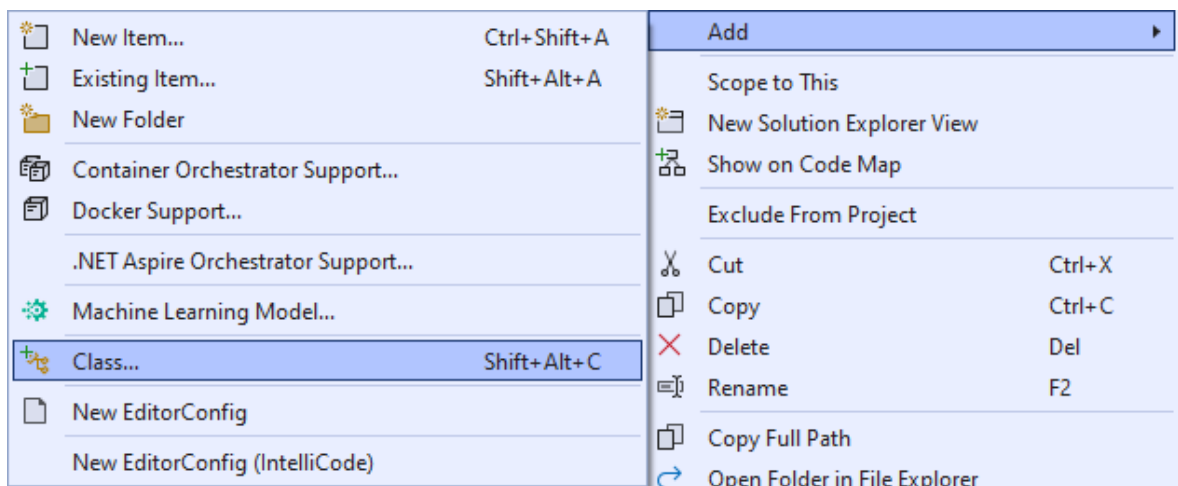
This is the second part of a complete walkthrough on how to use Windows Hello as an alternative to traditional username and password authentication systems in packaged Windows apps. This article picks up where Part 1, [Windows Hello login app](#), left off and extends the functionality to demonstrate how you can integrate Windows Hello into your existing application.

In order to build this project, you'll need some experience with C#, and XAML. You'll also need to be using Visual Studio 2022 on a Windows 10 or Windows 11 machine. See [Start developing Windows apps](#) for complete instructions on setting up your development environment.

Exercise 1: Server Side Logic

In this exercise, you start with the Windows Hello application built in the first lab and create a local mock server and database. This hands on lab is designed to teach how Windows Hello can be integrated into an existing system. By using a mock server and mock database, a lot of unrelated setup is eliminated. In your own applications, you will need to replace the mock objects with the real services and databases.

- To begin, open up the **WindowsHelloLogin** solution from the first [Windows Hello Hands On Lab](#).
- You will start by implementing the mock server and mock database. Create a new folder named "AuthService". In Solution Explorer, right-click the **WindowsHelloLogin** project and select **Add > New Folder**.
- Create **UserAccount** and **WindowsHelloDevices** classes that will act as models for data to be saved in the mock database. The **UserAccount** will be similar to the user model implemented on a traditional authentication server. Right-click the **AuthService** folder and add a new class named "UserAccount".



- Change the class scope to be public and add the following public properties for the **UserAccount** class. You will need to add a using statement for the `System.ComponentModel.DataAnnotations` namespace.

```
CS

using System;
using System.ComponentModel.DataAnnotations;

namespace WindowsHelloLogin.AuthService
{
    public class UserAccount
    {
        [Key, Required]
        public Guid UserId { get; set; }
        [Required]
        public string Username { get; set; }
    }
}
```

```

        public string Password { get; set; }
        // public List<WindowsHelloDevice> WindowsHelloDevices = new();
    }
}

```

You may have noticed the commented out list of **WindowsHelloDevices**. This is a modification you will need to make to an existing user model in your current implementation. The list of **WindowsHelloDevices** will contain a *deviceId*, the public key made from Windows Hello, and a [KeyCredentialAttestationResult](#). For this exercise, you will need to implement the *keyAttestationResult* as they are only provided by Windows Hello on devices that have a TPM (Trusted Platform Modules) chip. The **KeyCredentialAttestationResult** is a combination of multiple properties and would need to be split in order to save and load them with a database.

- Create a new class in the **AuthService** folder called "WindowsHelloDevice.cs". This is the model for the Windows Hello devices as discussed above. Change the class scope to be public and add the following properties.

CS

```

using System;

namespace WindowsHelloLogin.AuthService
{
    public class WindowsHelloDevice
    {
        // These are the new variables that will need to be added to
        // the existing UserAccount in the Database
        // The DeviceName is used to support multiple devices for the
        // one user.
        // This way the correct public key is easier to find as a new
        // public key is made for each device.
        // The KeyAttestationResult is only used if the User device has
        // a TPM (Trusted Platform Module) chip,
        // in most cases it will not. So will be left out for this
        // hands on lab.
        public Guid DeviceId { get; set; }
        public byte[] PublicKey { get; set; }
        // public KeyCredentialAttestationResult KeyAttestationResult {
        get; set; }
    }
}

```

- Return to UserAccount.cs and uncomment the list of Windows Hello devices.

CS

```

using System.Collections.Generic;

namespace WindowsHelloLogin.AuthService
{
    public class UserAccount
    {
        [Key, Required]
        public Guid UserId { get; set; }
        [Required]
        public string Username { get; set; }
        public string Password { get; set; }
        public List<WindowsHelloDevice> WindowsHelloDevices = new();
    }
}

```

- With the model for the **UserAccount** and the **WindowsHelloDevice** created, you need to create another new class in the **AuthService** folder that will act as the mock database, as this is a mock database from where you will be saving and loading a list of user accounts locally. In the real world, this would be your database implementation. Create a new class in the **AuthService** folder named "MockStore.cs". Change the class scope to public.
- As the mock store will save and load a list of user accounts locally, you can implement the logic to save and load that list using an **XmlSerializer**. You will also need to remember the filename and save location. In MockStore.cs implement the following:

```

CS

using System.Collections.Generic;
using System;
using System.IO;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Xml.Serialization;
using Windows.Storage;

namespace WindowsHelloLogin.AuthService
{
    public class MockStore
    {
        private const string USER_ACCOUNT_LIST_FILE_NAME =
"userAccountsList.txt";
        // This cannot be a const because the LocalFolder is accessed
at runtime
        private string _userAccountListPath = Path.Combine(
            ApplicationData.Current.LocalFolder.Path,
            USER_ACCOUNT_LIST_FILE_NAME);
    }
}

```

```

        private List<UserAccount> _mockDatabaseUserAccountsList;

#region Save and Load Helpers
    /// <summary>
    /// Create and save a useraccount list file. (Replacing the old
one)
    /// </summary>
    private async Task SaveAccountListAsync()
    {
        string accountsXml = SerializeAccountListToXml();

        if (File.Exists(_userAccountListPath))
        {
            StorageFile accountsFile = await
StorageFile.GetFileFromPathAsync(_userAccountListPath);
            await FileIO.WriteTextAsync(accountsFile, accountsXml);
        }
        else
        {
            StorageFile accountsFile = await
ApplicationData.Current.LocalFolder.CreateFileAsync(USER_ACCOUNT_LIST_F
ILE_NAME);
            await FileIO.WriteTextAsync(accountsFile, accountsXml);
        }
    }

    /// <summary>
    /// Gets the useraccount list file and deserializes it from XML
to a list of useraccount objects.
    /// </summary>
    /// <returns>List of useraccount objects</returns>
    private async Task LoadAccountListAsync()
    {
        if (File.Exists(_userAccountListPath))
        {
            StorageFile accountsFile = await
StorageFile.GetFileFromPathAsync(_userAccountListPath);

            string accountsXml = await
FileIO.ReadTextAsync(accountsFile);
            DeserializeXmlToAccountList(accountsXml);
        }

        // If the UserAccountList does not contain the sampleUser
Initialize the sample users
        // This is only needed as it in a Hand on Lab to
demonstrate a user being migrated.
        // In the real world, user accounts would just be in a
database.
        if (!_mockDatabaseUserAccountsList.Any(f =>
f.Username.Equals("sampleUsername")))
        {
            //If the list is empty, call InitializeSampleAccounts
and return the list
            //await InitializeSampleUserAccountsAsync();

```

```

    }
}

/// <summary>
/// Uses the local list of accounts and returns an XML
formatted string representing the list
/// </summary>
/// <returns>XML formatted list of accounts</returns>
private string SerializeAccountListToXml()
{
    var xmlizer = new XmlSerializer(typeof(List<UserAccount>));
    var writer = new StringWriter();
    xmlizer.Serialize(writer, _mockDatabaseUserAccountsList);
    return writer.ToString();
}

/// <summary>
/// Takes an XML formatted string representing a list of
accounts and returns a list object of accounts
/// </summary>
/// <param name="listAsXml">XML formatted list of
accounts</param>
/// <returns>List object of accounts</returns>
private List<UserAccount> DeserializeXmlToAccountList(string
listAsXml)
{
    var xmlizer = new XmlSerializer(typeof(List<UserAccount>));
    TextReader textreader = new StreamReader(new
MemoryStream(Encoding.UTF8.GetBytes(listAsXml)));
    return _mockDatabaseUserAccountsList =
(xmlizer.Deserialize(textreader)) as List<UserAccount>;
}
#endregion
}
}

```

- In the **LoadAccountListAsync** method, you may have noticed that an **InitializeSampleUserAccountsAsync** method was commented out. You will need to create this method in the **MockStore.cs**. This method will populate the user accounts list so that a login can take place. In the real world, the user database would already be populated. In this step, you will also be creating a constructor that will initialize the user list and call **LoadAccountListAsync**.

```

CS

namespace WindowsHelloLogin.AuthService
{
    public class MockStore
    {
        private const string USER_ACCOUNT_LIST_FILE_NAME =
"userAccountsList.txt";
        // This cannot be a const because the LocalFolder is accessed

```

```

at runtime
    private string _userAccountListPath = Path.Combine(
        ApplicationData.Current.LocalFolder.Path,
        USER_ACCOUNT_LIST_FILE_NAME);
    private List<UserAccount> _mockDatabaseUserAccountsList;

    public MockStore()
    {
        _mockDatabaseUserAccountsList = new List<UserAccount>();
        _ = LoadAccountListAsync();
    }

    private async Task InitializeSampleUserAccountsAsync()
    {
        // Create a sample Traditional User Account that only has a
        Username and Password
        // This will be used initially to demonstrate how to
        migrate to use Windows Hello

        var sampleUserAccount = new UserAccount()
        {
            UserId = Guid.NewGuid(),
            Username = "sampleUsername",
            Password = "samplePassword",
        };

        // Add the sampleUserAccount to the _mockDatabase
        _mockDatabaseUserAccountsList.Add(sampleUserAccount);
        await SaveAccountListAsync();
    }
}

```

- Now that the `InitializeSampleUserAccountsAsync` method exists uncomment the method call in the `LoadAccountListAsync` method.

```

CS

private async Task LoadAccountListAsync()
{
    if (File.Exists(_userAccountListPath))
    {
        StorageFile accountsFile = await
        StorageFile.GetFileFromPathAsync(_userAccountListPath);

        string accountsXml = await FileIO.ReadTextAsync(accountsFile);
        DeserializeXmlToAccountList(accountsXml);
    }

    // If the UserAccountList does not contain the sampleUser
    Initialize the sample users
    // This is only needed as it in a Hand on Lab to demonstrate a user
    migrating

```



```

        // In the real world user accounts would just be in a database
        if (!_mockDatabaseUserAccountsList.Any(f =>
            f.Username.Equals("sampleUsername")))
        {
            //If the list is empty InitializeSampleUserAccountsAsync and
            return the list
            await InitializeSampleUserAccountsAsync();
        }
    }
}

```

- The user accounts list in mock store can now be saved and loaded. Other parts of the application will need to have access to this list so there will need to be some methods to retrieve this data. Underneath the **InitializeSampleUserAccountsAsync** method, add the following methods to get data. They will allow you to get a user ID, a single user, a list of users for a specific Windows Hello device, and also get the public key for the user on a specific device.

CS

```

public Guid GetUserId(string username)
{
    if (_mockDatabaseUserAccountsList.Any())
    {
        UserAccount account =
        _mockDatabaseUserAccountsList.FirstOrDefault(f =>
            f.Username.Equals(username));
        if (account != null)
        {
            return account.UserId;
        }
    }
    return Guid.Empty;
}

public UserAccount GetUserAccount(Guid userId)
{
    return _mockDatabaseUserAccountsList.FirstOrDefault(f =>
        f.UserId.Equals(userId));
}

public List<UserAccount> GetUserAccountsForDevice(Guid deviceId)
{
    var usersForDevice = new List<UserAccount>();

    foreach (UserAccount account in _mockDatabaseUserAccountsList)
    {
        if (account.WindowsHelloDevices.Any(f =>
            f.DeviceId.Equals(deviceId)))
        {
            usersForDevice.Add(account);
        }
    }
}

```

```

        return usersForDevice;
    }

    public byte[] GetPublicKey(Guid userId, Guid deviceId)
    {
        UserAccount account =
            _mockDatabaseUserAccountsList.FirstOrDefault(f =>
                f.UserId.Equals(userId));
        if (account != null)
        {
            if (account.WindowsHelloDevices.Any())
            {
                return account.WindowsHelloDevices.FirstOrDefault(p =>
                    p.DeviceId.Equals(deviceId)).PublicKey;
            }
        }
        return null;
    }
}

```

- The next methods to implement will handle simple operations to add an account, remove an account, and also remove a device. Removing a device is needed as Windows Hello is device-specific. For each device to which you log in, a new public and private key pair will be created by Windows Hello. It's like having a different password for each device you sign in on, the only thing is you don't need to remember all those passwords; the server does. Add the following methods into the MockStore.cs.

```

CS

public async Task<UserAccount> AddAccountAsync(string username)
{
    UserAccount newAccount = null;
    try
    {
        newAccount = new UserAccount()
        {
            UserId = Guid.NewGuid(),
            Username = username,
        };

        _mockDatabaseUserAccountsList.Add(newAccount);
        await SaveAccountListAsync();
    }
    catch (Exception)
    {
        throw;
    }
    return newAccount;
}

```

```

public async Task<bool> RemoveAccountAsync(Guid userId)
{
    UserAccount userAccount = GetUserAccount(userId);
    if (userAccount != null)
    {
        _mockDatabaseUserAccountsList.Remove(userAccount);
        await SaveAccountListAsync();
        return true;
    }
    return false;
}

public async Task<bool> RemoveDeviceAsync(Guid userId, Guid deviceId)
{
    UserAccount userAccount = GetUserAccount(userId);
    WindowsHelloDevice deviceToRemove = null;
    if (userAccount != null)
    {
        foreach (WindowsHelloDevice device in
userAccount.WindowsHelloDevices)
        {
            if (device.DeviceId.Equals(deviceId))
            {
                deviceToRemove = device;
                break;
            }
        }
    }

    if (deviceToRemove != null)
    {
        //Remove the WindowsHelloDevice
        userAccount.WindowsHelloDevices.Remove(deviceToRemove);
        await SaveAccountListAsync();
    }

    return true;
}

```

- In the **MockStore** class add a method that will add Windows Hello related information to an existing **UserAccount**. This method will be called "WindowsHelloUpdateDetailsAsync" and will take parameters to identify the user, and the Windows Hello details. The **KeyAttestationResult** has been commented out when creating a WindowsHelloDevice, in a real world application you would require this.

CS

```

using System.Threading.Tasks;
using Windows.Security.Credentials;

public async Task WindowsHelloUpdateDetailsAsync(Guid userId, Guid

```

```

deviceId, byte[] publicKey,
    KeyCredentialAttestationResult keyAttestationResult)
{
    UserAccount existingUserAccount = GetUserAccount(userId);
    if (existingUserAccount != null)
    {
        if (!existingUserAccount.WindowsHelloDevices.Any(f =>
f.DeviceId.Equals(deviceId)))
        {
            existingUserAccount.WindowsHelloDevices.Add(new
WindowsHelloDevice()
            {
                DeviceId = deviceId,
                PublicKey = publicKey,
                // KeyAttestationResult = keyAttestationResult
            });
        }
    }
    await SaveAccountListAsync();
}

```

- The **MockStore** class is now complete, as this represents the database it should be considered private. In order to access the **MockStore**, an **AuthService** class is needed to manipulate the database data. In the **AuthService** folder create a new class called "AuthService.cs". Change the class scope to public and add a singleton instance pattern to make sure only one instance is ever created.

```

CS

namespace WindowsHelloLogin.AuthService
{
    public class AuthService
    {
        // Singleton instance of the AuthService
        // The AuthService is a mock of what a real world server and
        service implementation would be
        private static AuthService _instance;
        public static AuthService Instance
        {
            get
            {
                if (null == _instance)
                {
                    _instance = new AuthService();
                }
                return _instance;
            }
        }

        private AuthService()
        { }
    }
}

```

```
}  
}
```

- The **AuthService** class needs to create an instance of the **MockStore** class and provide access to the properties of the **MockStore** object.

CS

```
namespace WindowsHelloLogin.AuthService  
{  
    public class AuthService  
    {  
        //Singleton instance of the AuthService  
        //The AuthService is a mock of what a real world server and  
        database implementation would be  
        private static AuthService _instance;  
        public static AuthService Instance  
        {  
            get  
            {  
                if (null == _instance)  
                {  
                    _instance = new AuthService();  
                }  
                return _instance;  
            }  
        }  
  
        private AuthService()  
        { }  
  
        private MockStore _mockStore = new();  
  
        public Guid GetUserId(string username)  
        {  
            return _mockStore.GetUserId(username);  
        }  
  
        public UserAccount GetUserAccount(Guid userId)  
        {  
            return _mockStore.GetUserAccount(userId);  
        }  
  
        public List<UserAccount> GetUserAccountsForDevice(Guid  
deviceId)  
        {  
            return _mockStore.GetUserAccountsForDevice(deviceId);  
        }  
    }  
}
```

- You need methods in the **AuthService** class to access add, remove, and update Windows Hello details methods in the **MockStore** object. At the end of the **AuthService** class definition, add the following methods.

CS

```
using System.Threading.Tasks;
using Windows.Security.Credentials;

public async Task RegisterAsync(string username)
{
    await _mockStore.AddAccountAsync(username);
}

public async Task<bool> WindowsHelloRemoveUserAsync(Guid userId)
{
    return await _mockStore.RemoveAccountAsync(userId);
}

public async Task<bool> WindowsHelloRemoveDeviceAsync(Guid userId, Guid
deviceId)
{
    return await _mockStore.RemoveDeviceAsync(userId, deviceId);
}

public async Task WindowsHelloUpdateDetailsAsync(Guid userId, Guid
deviceId, byte[] publicKey,
    KeyCredentialAttestationResult keyAttestationResult)
{
    await _mockStore.WindowsHelloUpdateDetailsAsync(userId, deviceId,
publicKey, keyAttestationResult);
}
```

- The **AuthService** class needs to provide a method to validate credentials. This method will take a username and password and make sure that account exists and the password is valid. An existing system would have an equivalent method to this that checks the user is authorized. Add the following **ValidateCredentials** method to the **AuthService.cs** file.

CS

```
public bool ValidateCredentials(string username, string password)
{
    if (!string.IsNullOrEmpty(username) &&
    !string.IsNullOrEmpty(password))
    {
        // This would be used for existing accounts migrating to use
        Windows Hello
        Guid userId = GetUserId(username);
        if (userId != Guid.Empty)
```

```

    {
        UserAccount account = GetUserAccount(userId);
        if (account != null)
        {
            if (string.Equals(password, account.Password))
            {
                return true;
            }
        }
    }
    return false;
}

```

- The **AuthService** class needs a request challenge method that returns a challenge to the client to validate whether the user is who they claim to be. Then another method is needed in the **AuthService** class to receive the signed challenge back from the client. For this hands on lab, the method of how you determine if the signed challenge has been completed has been left incomplete. Every implementation of Windows Hello into an existing authentication system will be slightly different. The public key stored on the server needs to match with the result the client returned to the server. Add these two methods to AuthService.cs.

CS

```

using Windows.Security.Cryptography;
using Windows.Storage.Streams;

public IBuffer WindowsHelloRequestChallenge()
{
    return CryptographicBuffer.ConvertStringToBinary("ServerChallenge",
        BinaryStringEncoding.Utf8);
}

public bool SendServerSignedChallenge(Guid userId, Guid deviceId,
    byte[] signedChallenge)
{
    // Depending on your company polices and procedures this step will
    be different
    // It is at this point you will need to validate the
    signedChallenge that is sent back from the client.
    // Validation is used to ensure the correct user is trying to
    access this account.
    // The validation process will use the signedChallenge and the
    stored PublicKey
    // for the username and the specific device signin is called from.
    // Based on the validation result you will return a bool value to
    allow access to continue or to block the account.

    // For this sample validation will not happen as a best practice
    solution does not apply and will need to

```

```

        // be configured for each company.
        // Simply just return true.

        // You could get the User's Public Key with something similar to
        the following:
        byte[] userPublicKey = _mockStore.GetPublicKey(userId, deviceId);
        return true;
    }

```

Exercise 2: Client Side Logic

In this exercise, you will be changing the client side views and helper classes from the first lab to use the **AuthService** class. In the real world, the **AuthService** would be the authentication server and you would need to use Web API's to send and receive data from the server. For this hands on lab, the client and server are both local to keep things simple. The objective is to learn how to use the Windows Hello APIs.

- In the MainPage.xaml.cs, you can remove the **AccountHelper.LoadAccountListAsync** method call in the loaded method as the **AuthService** class creates an instance of the **MockStore** to load the accounts list. The **Loaded** method should now look like the snippet below. Note that the async method definition is removed as nothing is being awaited.

CS

```

private void MainPage_Loaded(object sender, RoutedEventArgs e)
{
    Frame.Navigate(typeof(UserSelection));
}

```

- Update the **Login** page interface to require a password be entered. This hands on lab demonstrates how an existing system could be migrated to use Windows Hello and existing accounts will have a username and a password. Also update the explanation at the bottom of the XAML to include the default password. Update the following XAML in Login.xaml.

XML

```

<Grid>
    <StackPanel>
        <TextBlock Text="Login" FontSize="36" Margin="4"
        TextAlignment="Center"/>

        <TextBlock x:Name="ErrorMessage" Text="" FontSize="20" Margin="4"
        Foreground="Red" TextAlignment="Center"/>
    
```



```

        <TextBlock Text="Enter your credentials below" Margin="0,0,0,20"
                TextWrapping="Wrap" Width="300"
                TextAlignment="Center" VerticalAlignment="Center"
FontSize="16"/>

        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
            <!-- Username Input -->
            <TextBlock x:Name="UserNameTextBlock" Text="Username: "
                FontSize="20" Margin="4" Width="100"/>
            <TextBox x:Name="UsernameTextBox"
PlaceholderText="sampleUsername" Width="200" Margin="4"/>
        </StackPanel>

        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
            <!-- Password Input -->
            <TextBlock x:Name="PasswordTextBlock" Text="Password: "
                FontSize="20" Margin="4" Width="100"/>
            <PasswordBox x:Name="PasswordBox"
PlaceholderText="samplePassword" Width="200" Margin="4"/>
        </StackPanel>

        <Button x:Name="LoginButton" Content="Login"
Background="DodgerBlue" Foreground="White"
                Click="LoginButton_Click" Width="80"
HorizontalAlignment="Center" Margin="0,20"/>

        <TextBlock Text="Don't have an account?"
                TextAlignment="Center" VerticalAlignment="Center"
FontSize="16"/>
        <TextBlock x:Name="RegisterButtonTextBlock" Text="Register now"

PointerPressed="RegisterButtonTextBlock_OnPointerPressed"
                Foreground="DodgerBlue"
                TextAlignment="Center" VerticalAlignment="Center"
FontSize="16"/>

        <Border x:Name="WindowsHelloStatus" Background="#22B14C"
                Margin="0,20" Height="100">
            <TextBlock x:Name="WindowsHelloStatusText" Text="Windows Hello is
ready to use!"
                Margin="4" TextAlignment="Center"
VerticalAlignment="Center" FontSize="20"/>
        </Border>

        <TextBlock x:Name="LoginExplanation" FontSize="24"
TextAlignment="Center" TextWrapping="Wrap"
                Text="Please Note: To demonstrate a login, validation
will only occur using the default username 'sampleUsername' and default
password 'samplePassword'"/>
    </StackPanel>
</Grid>

```

- In the code-behind file for the **Login** class, you will need to change the `Account` private variable at the top of the class to be a `UserAccount`. Change the `OnNavigateTo` event to cast the type to be a `UserAccount`. You will need the following using statement as well.

CS

```
using WindowsHelloLogin.AuthService;

namespace WindowsHelloLogin.Views
{
    public sealed partial class Login : Page
    {
        private UserAccount _account;
        private bool _isExistingAccount;

        public Login()
        {
            this.InitializeComponent();
        }

        protected override async void OnNavigatedTo(NavigationEventArgs e)
        {
            //Check Windows Hello is setup and available on this machine
            if (await WindowsHelloHelper.WindowsHelloAvailableCheckAsync())
            {
                if (e.Parameter != null)
                {
                    _isExistingAccount = true;
                    //Set the account to the existing account being passed in
                    _account = (UserAccount)e.Parameter;
                    UsernameTextBox.Text = _account.Username;
                    await SignInWindowsHelloAsync();
                }
            }

            private async void LoginButton_Click(object sender, RoutedEventArgs e)
            {
                ErrorMessage.Text = "";
                await SignInWindowsHelloAsync();
            }
        }
    }
}
```

- As the **Login** page is using a `UserAccount` object instead of the previous `Account` object, the `WindowsHelloHelper.cs` will need to be updated to use a `UserAccount` as a parameter type for some methods. You will need to change the following parameters for the **CreateWindowsHelloKeyAsync**, **RemoveWindowsHelloAccountAsync**, and **GetWindowsHelloAuthenticationMessageAsync** methods. As the `UserAccount` class has a `Guid` for a *UserId*, you will start using the `Id` in more places to be more precise.

CS

```
public static async Task<bool> CreateWindowsHelloKeyAsync(Guid userId,
string username)
{
    KeyCredentialRetrievalResult keyCreationResult = await
KeyCredentialManager.RequestCreateAsync(username,
KeyCredentialCreationOption.ReplaceExisting);

    return true;
}

public static async Task RemoveWindowsHelloAccountAsync(UserAccount
account)
{
}

public static async Task<bool>
GetWindowsHelloAuthenticationMessageAsync(UserAccount account)
{
    KeyCredentialRetrievalResult openKeyResult = await
KeyCredentialManager.OpenAsync(account.Username);
    //Calling OpenAsync will allow the user access to what is available
in the app and will not require user credentials again.
    //If you wanted to force the user to sign in again you can use the
following:
    //var consentResult = await
Windows.Security.Credentials.UI.UserConsentVerifier.RequestVerification
Async(account.Username);
    //This will ask for the either the password of the currently signed
in Microsoft Account or the PIN used for Windows Hello.

    if (openKeyResult.Status == KeyCredentialStatus.Success)
    {
        //If OpenAsync has succeeded, the next thing to think about is
whether the client application requires access to backend services.
        //If it does here you would Request a challenge from the
Server. The client would sign this challenge and the server
        //would check the signed challenge. If it is correct it would
allow the user access to the backend.
        //You would likely make a new method called RequestSignAsync to
handle all this
```

```

        //for example, RequestSignInAsync(openKeyResult);
        //Refer to the second Windows Hello sample for information on
        how to do this.

        //For this sample there is not concept of a server implemented
        so just return true.
        return true;
    }
    else if (openKeyResult.Status == KeyCredentialStatus.NotFound)
    {
        //If the _account is not found at this stage. It could be one
        of two errors.
        //1. Windows Hello has been disabled
        //2. Windows Hello has been disabled and re-enabled cause the
        Windows Hello Key to change.
        //Calling CreateWindowsHelloKeyAsync and passing through the
        account will attempt to replace the existing Windows Hello Key for that
        account.
        //If the error really is that Windows Hello is disabled then
        the CreateWindowsHelloKeyAsync method will output that error.
        if (await CreateWindowsHelloKeyAsync(account.UserId,
        account.Username))
        {
            //If the Windows Hello Key was again successfully created,
            Windows Hello has just been reset.
            //Now that the Windows Hello Key has been reset for the
            account retry sign in.
            return await
            GetWindowsHelloAuthenticationMessageAsync(account);
        }
    }

    // Can't use Windows Hello right now, try again later
    return false;
}

```

- The **SignInWindowsHelloAsync** method in Login.xaml.cs file will need to be updated to use the **AuthService** instead of the **AccountHelper**. Validation of credentials will happen through the **AuthService**. For this hands on lab, the only configured account is "sampleUsername". This account is created in the **InitializeSampleUserAccountsAsync** method in MockStore.cs. Update the **SignInWindowsHelloAsync** method in Login.xaml.cs now to reflect the code snippet below.

CS

```

private async Task SignInWindowsHelloAsync()
{
    if (_isExistingAccount)
    {
        if (await

```

```

WindowsHelloHelper.GetWindowsHelloAuthenticationMessageAsync(_account))
    {
        Frame.Navigate(typeof(Welcome), _account);
    }
}
else if
(AuthService.AuthService.Instance.ValidateCredentials(UsernameTextBox.Text, PasswordBox.Password))
{
    Guid userId =
AuthService.AuthService.Instance.GetUserId(UsernameTextBox.Text);

    if (userId != Guid.Empty)
    {
        //Now that the account exists on server try and create the
        necessary details and add them to the account
        if (await
WindowsHelloHelper.CreateWindowsHelloKeyAsync(userId,
UsernameTextBox.Text))
        {
            Debug.WriteLine("Successfully signed in with Windows
Hello!");

            //Navigate to the Welcome Screen.
            _account =
AuthService.AuthService.Instance.GetUserAccount(userId);
            Frame.Navigate(typeof(Welcome), _account);
        }
        else
        {
            //The Windows Hello account creation failed.
            //Remove the account from the server as the details
            were not configured
            await
AuthService.AuthService.Instance.WindowsHelloRemoveUserAsync(userId);

            ErrorMessage.Text = "Account Creation Failed";
        }
    }
}
else
{
    ErrorMessage.Text = "Invalid Credentials";
}
}

```

- As Windows Hello will create a different public and private key pair for each account on each device the **Welcome** page will need to display a list of registered devices for the logged in account and allow each one to be forgotten. In Welcome.xaml add in the following XAML underneath the `ForgetButton`. This will implement a forget device button, an error text area and a list to display all devices.

```

<Grid>
    <StackPanel>
        <TextBlock x:Name="Title" Text="Welcome" FontSize="40"
TextAlignment="Center"/>
        <TextBlock x:Name="UserNameText" FontSize="28"
TextAlignment="Center"/>

        <Button x:Name="BackToUserListButton" Content="Back to User List"
Click="Button_Restart_Click"
            HorizontalAlignment="Center" Margin="0,20"
Foreground="White" Background="DodgerBlue"/>

        <Button x:Name="ForgetButton" Content="Forget Me"
Click="Button_Forget_User_Click"
            Foreground="White"
            Background="Gray"
            HorizontalAlignment="Center"/>

        <Button x:Name="ForgetDeviceButton" Content="Forget Device"
Click="Button_Forget_Device_Click"
            Foreground="White"
            Background="Gray"
            Margin="0,40,0,20"
            HorizontalAlignment="Center"/>

        <TextBlock x:Name="ForgetDeviceErrorTextBlock" Text="Select a
device first"
            TextWrapping="Wrap" Width="300" Foreground="Red"
            TextAlignment="Center" VerticalAlignment="Center"
FontSize="16" Visibility="Collapsed"/>

        <ListView x:Name="UserListView" MaxHeight="500" MinWidth="350"
Width="350" HorizontalAlignment="Center">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <Grid Background="Gray" Height="50" Width="350"
HorizontalAlignment="Center" VerticalAlignment="Stretch" >
                        <TextBlock Text="{Binding DeviceId}"
HorizontalAlignment="Center" TextAlignment="Center"
VerticalAlignment="Center"
                            Foreground="White"/>
                    </Grid>
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>
    </StackPanel>
</Grid>

```

- In the Welcome.xaml.cs file, you need to change the private `Account` variable at the top of the class to be a private `UserAccount` variable. Then update the

`OnNavigatedTo` method to use the **AuthService** and retrieve information for the current account. When you have the account information you can set the `ItemsSource` of the list to display the devices. You will need to add a reference to the **AuthService** namespace.

```
CS

using WindowsHelloLogin.AuthService;

namespace WindowsHelloLogin.Views
{
    public sealed partial class Welcome : Page
    {
        private UserAccount _activeAccount;

        public Welcome()
        {
            InitializeComponent();
        }

        protected override void OnNavigatedTo(NavigationEventArgs e)
        {
            _activeAccount = (UserAccount)e.Parameter;
            if (_activeAccount != null)
            {
                UserAccount account =
                AuthService.AuthService.Instance.GetUserAccount(_activeAccount.UserId);
                if (account != null)
                {
                    UserListView.ItemsSource =
                    account.WindowsHelloDevices;
                    UserNameText.Text = account.Username;
                }
            }
        }
    }
}
```

- As you will be using the **AuthService** when removing an account, the reference to the **AccountHelper** in the `Button_Forget_User_Click` method can be removed. The method should now look as below.

```
CS

private async void Button_Forget_User_Click(object sender,
RoutedEventArgs e)
{
    //Remove it from Windows Hello
    await
    WindowsHelloHelper.RemoveWindowsHelloAccountAsync(_activeAccount);
}
```

```

        Debug.WriteLine($"User {_activeAccount.Username} deleted.");

        //Navigate back to UserSelection page.
        Frame.Navigate(typeof(UserSelection));
    }

```

- The **WindowsHelloHelper** method is not using the **AuthService** to remove the account. You need to make a call to the **AuthService** and pass the *userId*.

CS

```

public static async Task RemoveWindowsHelloAccountAsync(UserAccount
account)
{
    //Open the account with Windows Hello
    KeyCredentialRetrievalResult keyOpenResult = await
    KeyCredentialManager.OpenAsync(account.Username);

    if (keyOpenResult.Status == KeyCredentialStatus.Success)
    {
        // In the real world you would send key information to server
        to unregister
        await
        AuthService.AuthService.Instance.WindowsHelloRemoveUserAsync(account.User
erId);
    }

    //Then delete the account from the machines list of Windows Hello
    Accounts
    await KeyCredentialManager.DeleteAsync(account.Username);
}

```

- Before you can finish implementing the **Welcome** page, you need to create a method in **WindowsHelloHelper.cs** that will allow a device to be removed. Create a new method that will call **WindowsHelloRemoveDeviceAsync** in **AuthService**.

CS

```

public static async Task RemoveWindowsHelloDeviceAsync(UserAccount
account, Guid deviceId)
{
    await
    AuthService.AuthService.Instance.WindowsHelloRemoveDeviceAsync(account.
UserId, deviceId);
}

```

- In **Welcome.xaml.cs**, implement the **Button_Forget_Device_Click** event handler. This will use the selected device from the list of devices and use the Windows Hello

helper to call remove device. Remember to make the event handler async.

CS

```
private async void Button_Forget_Device_Click(object sender,
RoutedEventArgs e)
{
    WindowsHelloDevice selectedDevice = UserListView.SelectedItem as
WindowsHelloDevice;
    if (selectedDevice != null)
    {
        //Remove it from Windows Hello
        await
WindowsHelloHelper.RemoveWindowsHelloDeviceAsync(_activeAccount,
selectedDevice.DeviceId);

        Debug.WriteLine($"User {_activeAccount.Username} deleted.");

        if (!UserListView.Items.Any())
        {
            //Navigate back to UserSelection page.
            Frame.Navigate(typeof(UserSelection));
        }
    }
    else
    {
        ForgetDeviceErrorTextBlock.Visibility = Visibility.Visible;
    }
}
```

- The next page you will update is the **UserSelection** page. The **UserSelection** page will need to use the **AuthService** to retrieve all user accounts for the current device. Currently, there is no way for you get a device ID to pass to the **AuthService** so it can return user accounts for that device. In the **Utils** folder, create a new class called "Helpers.cs". Change the class scope to be public static and then add the following method that will allow you to retrieve the current device id.

CS

```
using System;
using Windows.Security.ExchangeActiveSyncProvisioning;

namespace WindowsHelloLogin.Utils
{
    public static class Helpers
    {
        public static Guid GetDeviceId()
        {
            //Get the Device ID to pass to the server
        }
    }
}
```

```

        var deviceInformation = new EasClientDeviceInformation();
        return deviceInformation.Id;
    }
}
}

```

- In the **UserSelection** page class, only the code-behind needs to change, not the user interface. In UserSelection.xaml.cs, update the **UserSelection_Loaded** method and the **UserSelectionChanged** method to use the **UserAccount** class instead of the **Account** class. You will also need to get all users for this device through the **AuthService**.

CS

```

using System.Linq;
using WindowsHelloLogin.AuthService;

namespace WindowsHelloLogin.Views
{
    public sealed partial class UserSelection : Page
    {
        public UserSelection()
        {
            InitializeComponent();
            Loaded += UserSelection_Loaded;
        }

        private void UserSelection_Loaded(object sender,
RoutedEventArgs e)
        {
            List<UserAccount> accounts =
AuthService.AuthService.Instance.GetUserAccountsForDevice(Helpers.GetDe
viceId());

            if (accounts.Any())
            {
                UserListView.ItemsSource = accounts;
                UserListView.SelectionChanged += UserSelectionChanged;
            }
            else
            {
                //If there are no accounts navigate to the Login page
                Frame.Navigate(typeof(Login));
            }
        }

        /// <summary>
        /// Function called when an account is selected in the list of
accounts
        /// Navigates to the Login page and passes the chosen account
        /// </summary>
        private void UserSelectionChanged(object sender,

```

```

RoutedEventArgs e)
{
    if (((ListView)sender).SelectedItem != null)
    {
        UserAccount account = (UserAccount)
        ((ListView)sender).SelectedItem;
        if (account != null)
        {
            Debug.WriteLine($"Account {account.Username}
selected!");
        }
        Frame.Navigate(typeof(Login), account);
    }
}
}
}

```

- The **WindowsHelloRegister** page needs to have the code-behind file updated. The user interface does not need any changes. In **WindowsHelloRegister.xaml.cs**, remove the private **Account** variable at the top of the class, as it's no longer needed. Update the **RegisterButton_Click_Async** event handler to use the **AuthService**. This method will create a new **UserAccount** and then try and update its account details. If Windows Hello fails to create a key, the account will be removed as the registration process failed.

CS

```

private async void RegisterButton_Click_Async(object sender,
RoutedEventArgs e)
{
    ErrorMessage.Text = "";

    //Validate entered credentials are acceptable
    if (!string.IsNullOrEmpty(UsernameTextBox.Text))
    {
        //Register an Account on the AuthService so that we can get
back a userId
        await
        AuthService.AuthService.Instance.RegisterAsync(UsernameTextBox.Text);
        Guid userId =
        AuthService.AuthService.Instance.GetUserId(UsernameTextBox.Text);

        if (userId != Guid.Empty)
        {
            //Now that the account exists on server try and create the
necessary details and add them to the account
            if (await
            WindowsHelloHelper.CreateWindowsHelloKeyAsync(userId,
            UsernameTextBox.Text))
            {
                //Navigate to the Welcome Screen.
            }
        }
    }
}

```

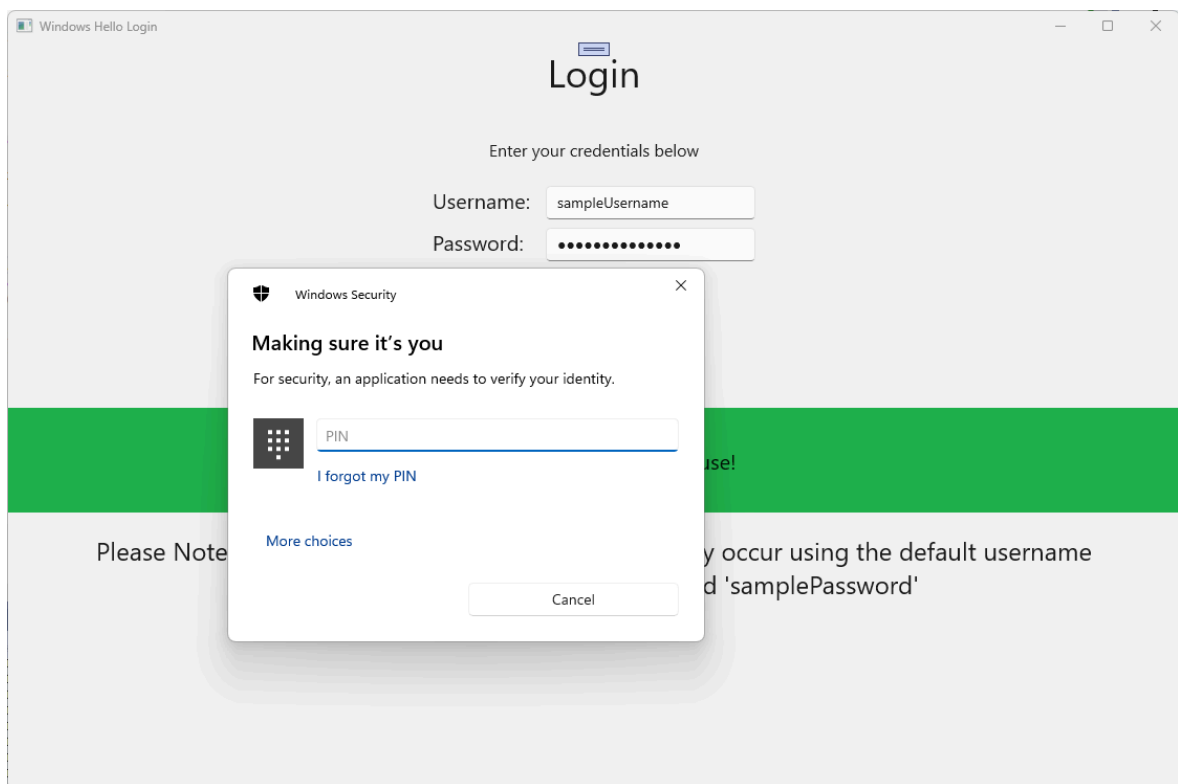
```

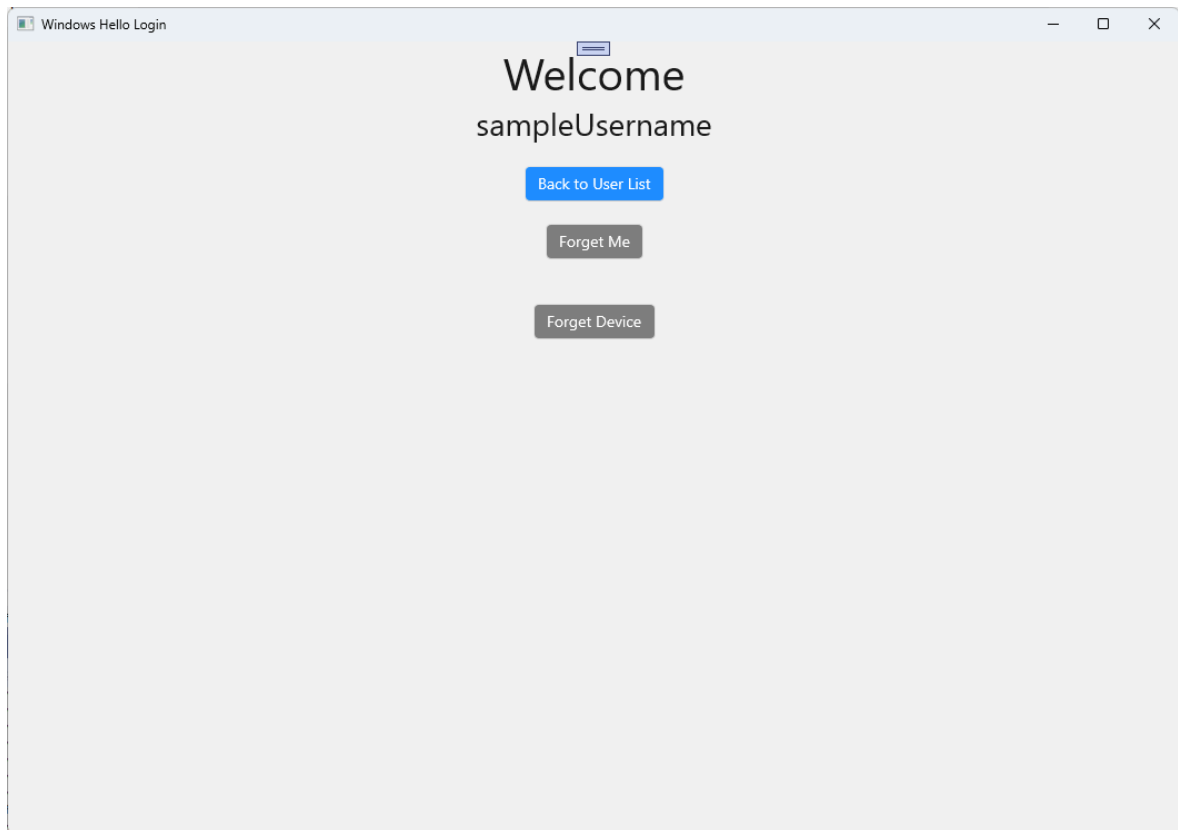
        Frame.Navigate(typeof(Welcome),
AuthService.AuthService.Instance.GetUserAccount(userId));
    }
    else
    {
        //The Windows Hello account creation failed.
        //Remove the account from the server as the details
were not configured
        await
AuthService.AuthService.Instance.WindowsHelloRemoveUserAsync(userId);

        ErrorMessage.Text = "Account Creation Failed";
    }
}
}
else
{
    ErrorMessage.Text = "Please enter a username";
}
}

```

- Build and run the application. Sign in to the sample user account with the credentials "sampleUsername" and "samplePassword". On the welcome screen, you may notice the Forget devices button is displayed, but there are no devices. When you are creating or migrating a user to work with Windows Hello the account information is not being pushed to the **AuthService**.





- To get the Windows Hello account information to the **AuthService**, the **WindowsHelloHelper.cs** will need to be updated. In the **CreateWindowsHelloKeyAsync** method, instead of only returning `true` in the case that's successful, you will need to call a new method which will try to get the **KeyAttestation**. While this hands on lab is not recording this information in the **AuthService**, you'll learn how you would get this information on the client side. Update the **CreateWindowsHelloKeyAsync** method as follows:

CS

```
public static async Task<bool> CreateWindowsHelloKeyAsync(Guid userId,
string username)
{
    KeyCredentialRetrievalResult keyCreationResult = await
    KeyCredentialManager.RequestCreateAsync(username,
    KeyCredentialCreationOption.ReplaceExisting);

    switch (keyCreationResult.Status)
    {
        case KeyCredentialStatus.Success:
            Debug.WriteLine("Successfully made key");
            await GetKeyAttestationAsync(userId, keyCreationResult);
            return true;
        case KeyCredentialStatus.UserCanceled:
            Debug.WriteLine("User cancelled sign-in process.");
            break;
        case KeyCredentialStatus.NotFound:
            // User needs to setup Windows Hello
            Debug.WriteLine($"Windows Hello is not set up!");
    }
}
```

```

{Environment.NewLine}Please go to Windows Settings and set up a PIN to
use it.");
        break;
    default:
        break;
    }

    return false;
}

```

- Create a **GetKeyAttestationAsync** method in WindowsHelloHelper.cs. This method will demonstrate how to obtain all the necessary information that can be provided by Windows Hello for each account on a specific device.

CS

```

using Windows.Storage.Streams;

private static async Task GetKeyAttestationAsync(Guid userId,
KeyCredentialRetrievalResult keyCreationResult)
{
    KeyCredential userKey = keyCreationResult.Credential;
    IBuffer publicKey = userKey.RetrievePublicKey();
    KeyCredentialAttestationResult keyAttestationResult = await
userKey.GetAttestationAsync();
    IBuffer keyAttestation = null;
    IBuffer certificateChain = null;
    bool keyAttestationIncluded = false;
    bool keyAttestationCanBeRetrievedLater = false;
    KeyCredentialAttestationStatus keyAttestationRetryType = 0;

    if (keyAttestationResult.Status ==
KeyCredentialAttestationStatus.Success)
    {
        keyAttestationIncluded = true;
        keyAttestation = keyAttestationResult.AttestationBuffer;
        certificateChain = keyAttestationResult.CertificateChainBuffer;
        Debug.WriteLine("Successfully made key and attestation");
    }
    else if (keyAttestationResult.Status ==
KeyCredentialAttestationStatus.TemporaryFailure)
    {
        keyAttestationRetryType =
KeyCredentialAttestationStatus.TemporaryFailure;
        keyAttestationCanBeRetrievedLater = true;
        Debug.WriteLine("Successfully made key but not attestation");
    }
    else if (keyAttestationResult.Status ==
KeyCredentialAttestationStatus.NotSupported)
    {
        keyAttestationRetryType =
KeyCredentialAttestationStatus.NotSupported;
        keyAttestationCanBeRetrievedLater = false;
    }
}

```

```

        Debug.WriteLine("Key created, but key attestation not
supported");
    }

    Guid deviceId = Helpers.GetDeviceId();

    //Update the Windows Hello details with the information we have
    just fetched above.
    //await UpdateWindowsHelloDetailsAsync(userId, deviceId,
    publicKey.ToArray(), keyAttestationResult);
}

```

- You may have noticed in the **GetKeyAttestationAsync** method that you just added the last line was commented out. This last line will be a new method you create that will send all the Windows Hello information to the **AuthService**. In the real world, you would need to send this to an actual server through a Web API.

CS

```

using System.Runtime.InteropServices.WindowsRuntime;
using System.Threading.Tasks;

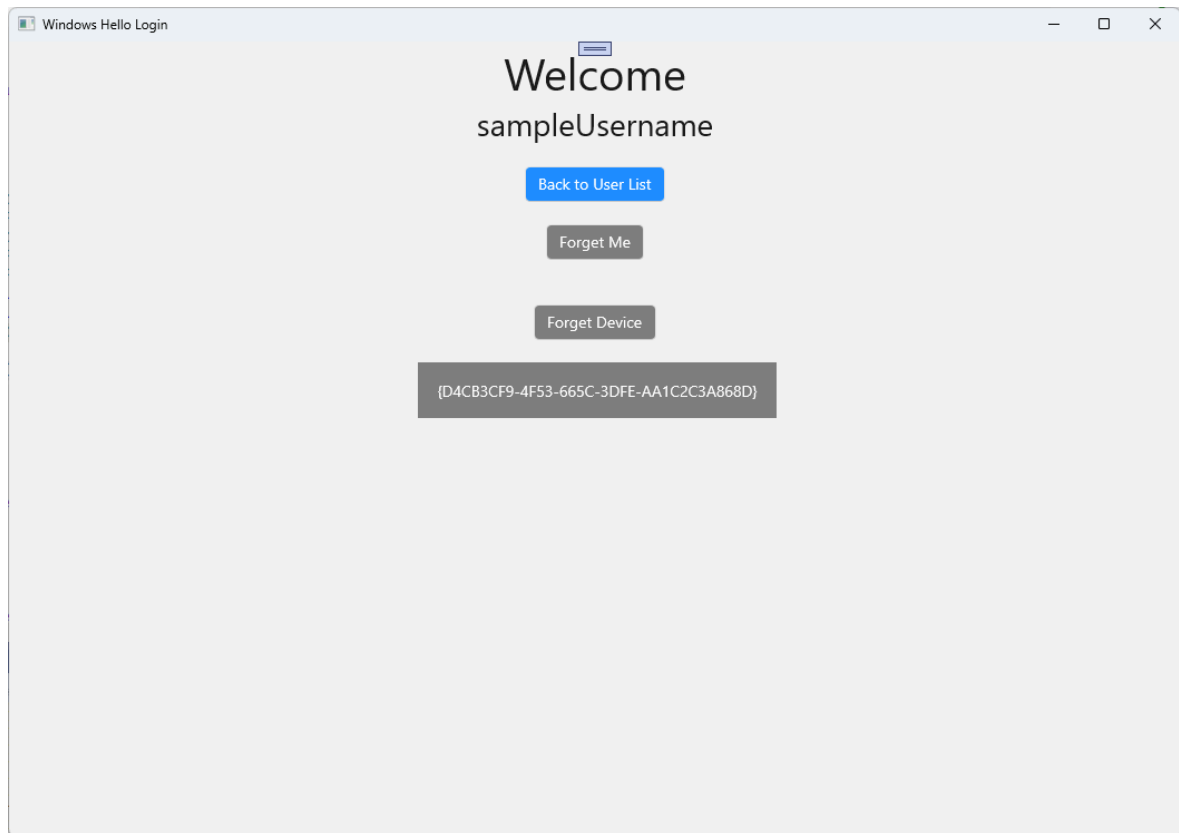
public static async Task<bool> UpdateWindowsHelloDetailsAsync(Guid
userId, Guid deviceId, byte[] publicKey, KeyCredentialAttestationResult
keyAttestationResult)
{
    //In the real world, you would use an API to add Windows Hello
    signing info to server for the signed in account.
    //For this tutorial, we do not implement a Web API for our server
    and simply mock the server locally.
    //The CreateWindowsHelloKey method handles adding the Windows Hello
    account locally to the device using the KeyCredential Manager

    //Using the userId the existing account should be found and
    updated.
    await
AuthService.AuthService.Instance.WindowsHelloUpdateDetailsAsync(userId,
deviceId, publicKey, keyAttestationResult);
    return true;
}

```

- Uncomment the last line in the **GetKeyAttestationAsync** method so that the Windows Hello information is being sent to the **AuthService**.
- Build and run the application and sign in with the default credentials as before. On the **Welcome** page, you will now see that the device Id is displayed. If you signed in on another device that would also be displayed here (if you had a cloud hosted auth service). For this hands on lab, the actual device Id is being displayed. In a real

implementation, you would want to display a friendly name that a person could understand and use to identify each device.



- To complete this hands on lab, you need a request and challenge for the user when they select from the user selection page and sign back in. The **AuthService** has two methods that you created to request a challenge, one that uses a signed challenge. In `WindowsHelloHelper.cs`, create a new method named **RequestSignAsync**. This will request a challenge from the **AuthService**, locally sign that challenge using a Windows Hello API and send the signed challenge to the **AuthService**. In this hands on lab, the **AuthService** will receive the signed challenge and return `true`. In an actual implementation, you would need to implement a verification mechanism to determine if the challenge was signed by the correct user on the correct device. Add the method below to the `WindowsHelloHelper.cs`

CS

```
private static async Task<bool> RequestSignAsync(Guid userId,
KeyCredentialRetrievalResult openKeyResult)
{
    // Calling userKey.RequestSignAsync() prompts the uses to enter the
    PIN or use Biometrics (Windows Hello).
    // The app would use the private key from the user account to sign
    the sign-in request (challenge)
    // The client would then send it back to the server and await the
    servers response.
    IBuffer challengeMessage =
```



```

AuthService.AuthService.Instance.WindowsHelloRequestChallenge();
    KeyCredential userKey = openKeyResult.Credential;
    KeyCredentialOperationResult signResult = await
userKey.RequestSignAsync(challengeMessage);

    if (signResult.Status == KeyCredentialStatus.Success)
    {
        // If the challenge from the server is signed successfully
        // send the signed challenge back to the server and await the
servers response
        return
AuthService.AuthService.Instance.SendServerSignedChallenge(
        userId, Helpers.GetDeviceId(),
signResult.Result.ToArray());
    }
    else if (signResult.Status == KeyCredentialStatus.UserCanceled)
    {
        // User cancelled the Windows Hello PIN entry.
    }
    else if (signResult.Status == KeyCredentialStatus.NotFound)
    {
        // Must recreate Windows Hello key
    }
    else if (signResult.Status ==
KeyCredentialStatus.SecurityDeviceLocked)
    {
        // Can't use Windows Hello right now, remember that hardware
failed and suggest restart
    }
    else if (signResult.Status == KeyCredentialStatus.UnknownError)
    {
        // Can't use Windows Hello right now, try again later
    }

    return false;
}

```

- In the **WindowsHelloHelper** class, call the **RequestSignAsync** method from the **GetWindowsHelloAuthenticationMessageAsync** method.

CS

```

public static async Task<bool>
GetWindowsHelloAuthenticationMessageAsync(UserAccount account)
{
    KeyCredentialRetrievalResult openKeyResult = await
KeyCredentialManager.OpenAsync(account.Username);
    // Calling OpenAsync will allow the user access to what is
available in the app and will not require user credentials again.
    // If you wanted to force the user to sign in again you can use the
following:
    // var consentResult = await
Windows.Security.Credentials.UI.UserConsentVerifier.RequestVerification

```

```

Async(account.Username);
    // This will ask for the either the password of the currently
    signed in Microsoft Account or the PIN used for Windows Hello.

    if (openKeyResult.Status == KeyCredentialStatus.Success)
    {
        //If OpenAsync has succeeded, the next thing to think about is
        whether the client application requires access to backend services.
        //If it does here you would Request a challenge from the
        Server. The client would sign this challenge and the server
        //would check the signed challenge. If it is correct it would
        allow the user access to the backend.
        //You would likely make a new method called RequestSignAsync to
        handle all this
        //for example, RequestSignAsync(openKeyResult);
        //Refer to the second Windows Hello sample for information on
        how to do this.

        return await RequestSignAsync(account.UserId, openKeyResult);
    }
    else if (openKeyResult.Status == KeyCredentialStatus.NotFound)
    {
        //If the _account is not found at this stage. It could be one
        of two errors.
        //1. Windows Hello has been disabled
        //2. Windows Hello has been disabled and re-enabled cause the
        Windows Hello Key to change.
        //Calling CreateWindowsHelloKeyAsync and passing through the
        account will attempt to replace the existing Windows Hello Key for that
        account.
        //If the error really is that Windows Hello is disabled then
        the CreateWindowsHelloKeyAsync method will output that error.
        if (await CreateWindowsHelloKeyAsync(account.UserId,
        account.Username))
        {
            //If the Windows Hello Key was again successfully created,
            Windows Hello has just been reset.
            //Now that the Windows Hello Key has been reset for the
            _account retry sign in.
            return await
            GetWindowsHelloAuthenticationMessageAsync(account);
        }
    }

    // Can't use Windows Hello right now, try again later
    return false;
}

```

- Throughout this exercise, you have updated the client side application to use the **AuthService**. By doing this, you have been able to eliminate the need for the **Account** class and the **AccountHelper** class. Delete the **Account** class, the **Models** folder, and the **AccountHelper** class in the **Utils** folder. You'll need to remove all

reference to the `WindowsHelloLogin.Models` namespace throughout the application before the solution will successfully build.

- Build and run the application and enjoy using Windows Hello with the mock service and database.

In this hands on lab, you have learned how to use the Windows Hello APIs to replace the need for passwords when using authentication from a Windows machine. When you consider how much energy is expended by people maintaining passwords and supporting lost passwords in existing systems, you should see the benefit of moving to this new Windows Hello system of authentication.

We have left as an exercise for you the details of how you'll implement the authentication on the service and server side. It's expected that most developers will have existing systems that will need to be migrated to start working with Windows Hello. The details of each of these system will differ.

Related topics

- [Windows Hello](#)
- [Windows Hello login app](#)

Smart cards

Article • 08/09/2024

This topic explains how Windows apps can use smart cards to connect users to secure network services, including how to access physical smart card readers, create virtual smart cards, communicate with smart cards, authenticate users, reset user PINs, and remove or disconnect smart cards.

The Windows Runtime (WinRT) APIs for smart cards are part of the [Windows Software Development Kit \(SDK\)](#). These APIs were created for use in Universal Windows Platform (UWP) apps, but they can also be used in WinUI apps or in packaged desktop apps, including WPF and Windows Forms. For more information about using WinRT APIs in your Windows desktop app, see [Call Windows Runtime APIs in desktop apps](#).

Configure the app manifest

Before your app can authenticate users using smart cards or virtual smart cards, you must set the **Shared User Certificates** capability in the project Package.appxmanifest file of your WinUI project or packaging project.

Access connected card readers and smart cards

You can query for readers and attached smart cards by passing the device ID (specified in [DeviceInformation](#)) to the [SmartCardReader.FromIdAsync](#) method. To access the smart cards currently attached to the returned reader device, call [SmartCardReader.FindAllCardsAsync](#).

CS

```
string selector = SmartCardReader.GetDeviceSelector();
DeviceInformationCollection devices =
    await DeviceInformation.FindAllAsync(selector);

foreach (DeviceInformation device in devices)
{
    SmartCardReader reader =
        await SmartCardReader.FromIdAsync(device.Id);

    // For each reader, we want to find all the cards associated
    // with it. Then we will create a SmartCardListItem for
    // each (reader, card) pair.
    IReadOnlyList<SmartCard> cards =
```

```
        await reader.FindAllCardsAsync();  
    }
```

You should also enable your app to observe for [CardAdded](#) events by implementing a method to handle app behavior on card insertion.

```
cs  
  
private void reader_CardAdded(SmartCardReader sender, CardAddedEventArgs  
args)  
{  
    // A card has been inserted into the sender SmartCardReader.  
}
```

You can then pass each returned [SmartCard](#) object to [SmartCardProvisioning](#) to access the methods that allow your app to access and customize its configuration.

Create a virtual smart card

To create a virtual smart card using [SmartCardProvisioning](#), your app will first need to provide a friendly name, an admin key, and a [SmartCardPinPolicy](#). The friendly name is generally something provided to the app, but your app will still need to provide an admin key and generate an instance of the current **SmartCardPinPolicy** before passing all three values to [RequestVirtualSmartCardCreationAsync](#).

1. Create a new instance of a [SmartCardPinPolicy](#)
2. Generate the admin key value by calling [CryptographicBuffer.GenerateRandom](#) on the admin key value provided by the service or management tool.
3. Pass these values along with the *FriendlyNameText* string to [RequestVirtualSmartCardCreationAsync](#).

```
cs  
  
var pinPolicy = new SmartCardPinPolicy  
{  
    MinLength = 6  
};  
  
IBuffer adminkey = CryptographicBuffer.GenerateRandom(24);  
  
SmartCardProvisioning provisioning = await  
    SmartCardProvisioning.RequestVirtualSmartCardCreationAsync(  
        "Card friendly name",  
        adminkey,  
        pinPolicy);
```

Once [RequestVirtualSmartCardCreationAsync](#) has returned the associated [SmartCardProvisioning](#) object, the virtual smart card is provisioned and ready for use.

ⓘ Note

In order to create a virtual smart card using a packaged Windows app, the user running the app must be a member of the administrators group. If the user is not a member of the administrators group, virtual smart card creation will fail.

Handle authentication challenges

To authenticate with smart cards or virtual smart cards, your app must provide the behavior to complete challenges between the admin key data stored on the card, and the admin key data maintained by the authentication server or management tool.

The following code shows how to support smart card authentication for services or modification of physical or virtual card details. If the data generated using the admin key on the card ("challenge") is the same as the admin key data provided by the server or management tool ("adminkey"), authentication is successful.

CS

```
static class ChallengeResponseAlgorithm
{
    public static IBuffer CalculateResponse(IBuffer challenge, IBuffer
adminkey)
    {
        if (challenge == null)
            throw new ArgumentNullException("challenge");
        if (adminkey == null)
            throw new ArgumentNullException("adminkey");

        SymmetricKeyAlgorithmProvider objAlg =
SymmetricKeyAlgorithmProvider.OpenAlgorithm(SymmetricAlgorithmNames.TripleDe
sCbc);
        var symmetricKey = objAlg.CreateSymmetricKey(adminkey);
        var buffEncrypted = CryptographicEngine.Encrypt(symmetricKey,
challenge, null);
        return buffEncrypted;
    }
}
```

You will see this code referenced throughout the remainder of this topic as we review how to complete an authentication action, and how to apply changes to smart card and virtual smart card information.

Verify smart card or virtual smart card authentication response

Now that we have the logic for authentication challenges defined, we can communicate with the reader to access the smart card, or alternatively, access a virtual smart card for authentication.

1. To begin the challenge, call [GetChallengeContextAsync](#) from the [SmartCardProvisioning](#) object associated with the smart card. This will generate an instance of [SmartCardChallengeContext](#), which contains the card's [Challenge](#) value.
2. Next, pass the card's challenge value and the admin key provided by the service or management tool to the **ChallengeResponseAlgorithm** that we defined in the previous example.
3. [VerifyResponseAsync](#) will return **true** if authentication is successful.

CS

```
bool verifyResult = false;
SmartCard card = await rootPage.GetSmartCard();
SmartCardProvisioning provisioning =
    await SmartCardProvisioning.FromSmartCardAsync(card);

SmartCardChallengeContext context =
    await provisioning.GetChallengeContextAsync();

IBuffer response = ChallengeResponseAlgorithm.CalculateResponse(
    context.Challenge,
    rootPage.AdminKey);

verifyResult = await context.VerifyResponseAsync(response);
```

Change or reset a user PIN

To change the PIN associated with a smart card:

1. Access the card and generate the associated [SmartCardProvisioning](#) object.
2. Call [RequestPinChangeAsync](#) to display a UI to the user to complete this operation.
3. If the PIN was successfully changed the call will return **true**.

CS

```
SmartCardProvisioning provisioning =
    await SmartCardProvisioning.FromSmartCardAsync(card);
```

```
bool result = await provisioning.RequestPinChangeAsync();
```

To request a PIN reset:

1. Call [RequestPinResetAsync](#) to initiate the operation. This call includes a [SmartCardPinResetHandler](#) method that represents the smart card and the pin reset request.
2. [SmartCardPinResetHandler](#) provides information that our [ChallengeResponseAlgorithm](#), wrapped in a [SmartCardPinResetDeferral](#) call, uses to compare the card's challenge value and the admin key provided by the service or management tool to authenticate the request.
3. If the challenge is successful, the [RequestPinResetAsync](#) call is completed; returning **true** if the PIN was successfully reset.

CS

```
SmartCardProvisioning provisioning =  
    await SmartCardProvisioning.FromSmartCardAsync(card);  
  
bool result = await provisioning.RequestPinResetAsync(  
    (pinResetSender, request) =>  
    {  
        SmartCardPinResetDeferral deferral =  
            request.GetDeferral();  
  
        try  
        {  
            IBuffer response =  
                ChallengeResponseAlgorithm.CalculateResponse(  
                    request.Challenge,  
                    rootPage.AdminKey);  
            request.SetResponse(response);  
        }  
        finally  
        {  
            deferral.Complete();  
        }  
    }  
));  
}
```

Remove a smart card or virtual smart card

When a physical smart card is removed a [CardRemoved](#) event will fire when the card is deleted.

Associate the firing of this event with the card reader with the method that defines your app's behavior on card or reader removal as an event handler. This behavior can be something as simply as providing notification to the user that the card was removed.

CS

```
reader = card.Reader;  
reader.CardRemoved += HandleCardRemoved;
```

The removal of a virtual smart card is handled programmatically by first retrieving the card and then calling [RequestVirtualSmartCardDeletionAsync](#) from the [SmartCardProvisioning](#) returned object.

CS

```
bool result = await SmartCardProvisioning  
    .RequestVirtualSmartCardDeletionAsync(card);
```

Related content

- [Call Windows Runtime APIs in desktop apps](#)
- [SmartCard](#)

Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



Windows developer feedback

Windows developer is an open source project. Select a link to provide feedback:



[Open a documentation issue](#)



[Provide product feedback](#)

Share certificates between apps

Article • 10/20/2022

Universal Windows Platform (UWP) apps that require secure authentication beyond a user Id and password combination can use certificates for authentication. Certificate authentication provides a high level of trust when authenticating a user. In some cases, a group of services will want to authenticate a user for multiple apps. This article shows how you can authenticate multiple apps using the same certificate, and how you can provide convenient code for a user to import a certificate that was provided to access secured web services.

Apps can authenticate to a web service using a certificate, and multiple apps can use a single certificate from the certificate store to authenticate the same user. If a certificate does not exist in the store, you can add code to your app to import a certificate from a PFX file.

Enable Microsoft Internet Information Services (IIS) and client certificate mapping

This article uses Microsoft Internet Information Services (IIS) for example purposes. IIS is not enabled by default. You can enable IIS by using the Control Panel.

1. Open the Control Panel and select **Programs**.
2. Select **Turn Windows features on or off**.
3. Expand **Internet Information Services** and then expand **World Wide Web Services**. Expand **Application Development Features** and select **ASP.NET 3.5** and **ASP.NET 4.5**. Making these selections will automatically enable **Internet Information Services**.
4. Click **OK** to apply the changes.

Create and publish a secured web service

1. Run Microsoft Visual Studio as administrator and select **New Project** from the start page. Administrator access is required to publish a web service to an IIS server. In the New Project dialog, change the framework to **.NET Framework 3.5**. Select **Visual C# -> Web -> Visual Studio -> ASP.NET Web Service Application**. Name the application "FirstContosoBank". Click **OK** to create the project.
2. In the **Service1.asmx.cs** file, replace the default **HelloWorld** web method with the following "Login" method.

CS

```
[WebMethod]
public string Login()
{
    // Verify certificate with CA
    var cert = new
System.Security.Cryptography.X509Certificates.X509Certificate2(
    this.Context.Request.ClientCertificate.Certificate);
    bool test = cert.Verify();
    return test.ToString();
}
```

3. Save the **Service1.asmx.cs** file.
4. In the **Solution Explorer**, right-click the "FirstContosoBank" app and select **Publish**.
5. In the **Publish Web** dialog, create a new profile and name it "ContosoProfile". Click **Next**.
6. On the next page, enter the server name for your IIS server, and specify a site name of "Default Web Site/FirstContosoBank". Click **Publish** to publish your web service.

Configure your web service to use client certificate authentication

1. Run the **Internet Information Services (IIS) Manager**.
2. Expand the sites for your IIS server. Under the **Default Web Site**, select the new "FirstContosoBank" web service. In the **Actions** section, select **Advanced Settings....**
3. Set the **Application Pool** to **.NET v2.0** and click **OK**.
4. In the **Internet Information Services (IIS) Manager**, select your IIS server and then double-click **Server Certificates**. In the **Actions** section, select **Create Self-Signed Certificate....** Enter "ContosoBank" as the friendly name for the certificate and click **OK**. This will create a new certificate for use by the IIS server in the form of "<server-name>.<domain-name>".
5. In the **Internet Information Services (IIS) Manager**, select the default web site. In the **Actions** section, select **Binding** and then click **Add....** Select "https" as the type, set the port to "443", and enter the full host name for your IIS server ("<server-name>.<domain-name>"). Set the SSL certificate to "ContosoBank". Click **OK**. Click **Close** in the **Site Bindings** window.
6. In the **Internet Information Services (IIS) Manager**, select the "FirstContosoBank" web service. Double-click **SSL Settings**. Check **Require SSL**. Under **Client certificates**, select **Require**. In the **Actions** section, click **Apply**.

7. You can verify that the web service is configured correctly by opening your web browser and entering the following web address: "https://<server-name>.<domain-name>/FirstContosoBank/Service1.asmx". For example, "https://myserver.example.com/FirstContosoBank/Service1.asmx". If your web service is configured correctly, you will be prompted to select a client certificate in order to access the web service.

You can repeat the previous steps to create multiple web services that can be accessed using the same client certificate.

Create a UWP app that uses certificate authentication

Now that you have one or more secured web services, your apps can use certificates to authenticate to those web services. When you make a request to an authenticated web service using the [HttpClient](#) object, the initial request will not contain a client certificate. The authenticated web service will respond with a request for client authentication. When this occurs, the Windows client will automatically query the certificate store for available client certificates. Your user can select from these certificates to authenticate to the web service. Some certificates are password protected, so you will need to provide the user with a way to input the password for a certificate.

If there are no client certificates available, then the user will need to add a certificate to the certificate store. You can include code in your app that enables a user to select a PFX file that contains a client certificate and then import that certificate into the client certificate store.

Tip You can use `makecert.exe` to create a PFX file to use with this quickstart. For information on using `makecert.exe`, see [MakeCert](#).

1. Open Visual Studio and create a new project from the start page. Name the new project "FirstContosoBankApp". Click **OK** to create the new project.
2. In the `MainPage.xaml` file, add the following XAML to the default **Grid** element. This XAML includes a button to browse for a PFX file to import, a text box to enter a password for a password-protected PFX file, a button to import a selected PFX file, a button to log in to the secured web service, and a text block to display the status of the current action.

XML

```

<Button x:Name="Import" Content="Import Certificate (PFX file)"
HorizontalAlignment="Left" Margin="352,305,0,0" VerticalAlignment="Top"
Height="77" Width="260" Click="Import_Click" FontSize="16"/>
<Button x:Name="Login" Content="Login" HorizontalAlignment="Left"
Margin="611,305,0,0" VerticalAlignment="Top" Height="75" Width="240"
Click="Login_Click" FontSize="16"/>
<TextBlock x:Name="Result" HorizontalAlignment="Left"
Margin="355,398,0,0" TextWrapping="Wrap" VerticalAlignment="Top"
Height="153" Width="560"/>
<PasswordBox x:Name="PfxPassword" HorizontalAlignment="Left"
Margin="483,271,0,0" VerticalAlignment="Top" Width="229"/>
<TextBlock HorizontalAlignment="Left" Margin="355,271,0,0"
TextWrapping="Wrap" Text="PFX password" VerticalAlignment="Top"
FontSize="18" Height="32" Width="123"/>
<Button x:Name="Browse" Content="Browse for PFX file"
HorizontalAlignment="Left" Margin="352,189,0,0" VerticalAlignment="Top"
Click="Browse_Click" Width="499" Height="68" FontSize="16"/>
<TextBlock HorizontalAlignment="Left" Margin="717,271,0,0"
TextWrapping="Wrap" Text="(Optional)" VerticalAlignment="Top"
Height="32" Width="83" FontSize="16"/>

```

3. Save the MainPage.xaml file.

4. In the MainPage.xaml.cs file, add the following using statements.

CS

```

using Windows.Web.Http;
using System.Text;
using Windows.Security.Cryptography.Certificates;
using Windows.Storage.Pickers;
using Windows.Storage;
using Windows.Storage.Streams;

```

5. In the MainPage.xaml.cs file, add the following variables to the **MainPage** class. They specify the address for the secured "Login" method of your "FirstContosoBank" web service, and a global variable that holds a PFX certificate to import into the certificate store. Update the <server-name> to the fully-qualified server name for your Microsoft Internet Information Server (IIS) server.

CS

```

private Uri requestUri = new Uri("https://<server-
name>/FirstContosoBank/Service1.asmx?op=Login");
private string pfxCert = null;

```

6. In the MainPage.xaml.cs file, add the following click handler for the login button and method to access the secured web service.

CS

```
private void Login_Click(object sender, RoutedEventArgs e)
{
    MakeHttpsCall();
}

private async void MakeHttpsCall()
{
    StringBuilder result = new StringBuilder("Login ");
    HttpResponseMessage response;
    try
    {
        Windows.Web.Http.HttpClient httpClient = new
Windows.Web.Http.HttpClient();
        response = await httpClient.GetAsync(requestUri);
        if (response.StatusCode == HttpStatusCode.Ok)
        {
            result.Append("successful");
        }
        else
        {
            result = result.Append("failed with ");
            result = result.Append(response.StatusCode);
        }
    }
    catch (Exception ex)
    {
        result = result.Append("failed with ");
        result = result.Append(ex.Message);
    }

    Result.Text = result.ToString();
}
```

7. In the MainPage.xaml.cs file, add the following click handlers for the button to browse for a PFX file and the button to import a selected PFX file into the certificate store.

CS

```
private async void Import_Click(object sender, RoutedEventArgs e)
{
    try
    {
        Result.Text = "Importing selected certificate into user
certificate store....";
        await
CertificateEnrollmentManager.UserCertificateEnrollmentManager.ImportPfx
DataAsync(
            pfxCert,
```

```

        PfxPassword.Password,
        ExportOption.Exportable,
        KeyProtectionLevel.NoConsent,
        InstallOptions.DeleteExpired,
        "Import Pfx");

        Result.Text = "Certificate import succeeded";
    }
    catch (Exception ex)
    {
        Result.Text = "Certificate import failed with " + ex.Message;
    }
}

private async void Browse_Click(object sender, RoutedEventArgs e)
{
    StringBuilder result = new StringBuilder("Pfx file selection ");
    FileOpenPicker pfxFilePicker = new FileOpenPicker();
    pfxFilePicker.FileTypeFilter.Add(".pfx");
    pfxFilePicker.CommitButtonText = "Open";
    try
    {
        StorageFile pfxFile = await
pfxFilePicker.PickSingleFileAsync();
        if (pfxFile != null)
        {
            IBuffer buffer = await FileIO.ReadBufferAsync(pfxFile);
            using (DataReader dataReader =
DataReader.FromBuffer(buffer))
            {
                byte[] bytes = new byte[buffer.Length];
                dataReader.ReadBytes(bytes);
                pfxCert = System.Convert.ToBase64String(bytes);
                PfxPassword.Password = string.Empty;
                result.Append("succeeded");
            }
        }
        else
        {
            result.Append("failed");
        }
    }
    catch (Exception ex)
    {
        result.Append("failed with ");
        result.Append(ex.Message); ;
    }

    Result.Text = result.ToString();
}

```

8. Run your app and log in to your secured web service as well as import a PFX file into the local certificate store.

You can use these steps to create multiple apps that use the same user certificate to access the same or different secured web services.

Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Web Account Manager

Article • 10/20/2022

This article describes how to use the [AccountsSettingsPane](#) to connect your Universal Windows Platform (UWP) app to external identity providers, like Microsoft or Facebook, using the Windows 10 and Windows 11 Web Account Manager APIs. You'll learn how to request a user's permission to use their Microsoft account, obtain an access token, and use it to perform basic operations (like get profile data or upload files to their OneDrive account). The steps are similar for getting user permission and access with any identity provider that supports the Web Account Manager.

❗ Note

For a complete code sample, see the [WebAccountManagement sample on GitHub](#) [↗](#).

Get set up

First, create a new, blank app in Visual Studio.

Second, in order to connect to identity providers, you'll need to associate your app with the Store. To do this, right click your project, choose **Store/Publish > Associate app with the store**, and follow the wizard's instructions.

Third, create a very basic UI consisting of a simple XAML button and two text boxes.

XML

```
<StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
    <Button x:Name="LoginButton" Content="Log in" Click="LoginButton_Click"
/>
    <TextBlock x:Name="UserIdTextBlock"/>
    <TextBlock x:Name="UserNameTextBlock"/>
</StackPanel>
```

And an event handler attached to your button in the code-behind:

C#

```
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
}
```

Lastly, add the following namespaces so you don't have to worry about any reference issues later:

C#

```
using System;
using Windows.Security.Authentication.Web.Core;
using Windows.System;
using Windows.UI.ApplicationSettings;
using Windows.UI.Xaml;
using Windows.UI.Xaml.Controls;
using Windows.Data.Json;
using Windows.UI.Xaml.Navigation;
using Windows.Web.Http;
```

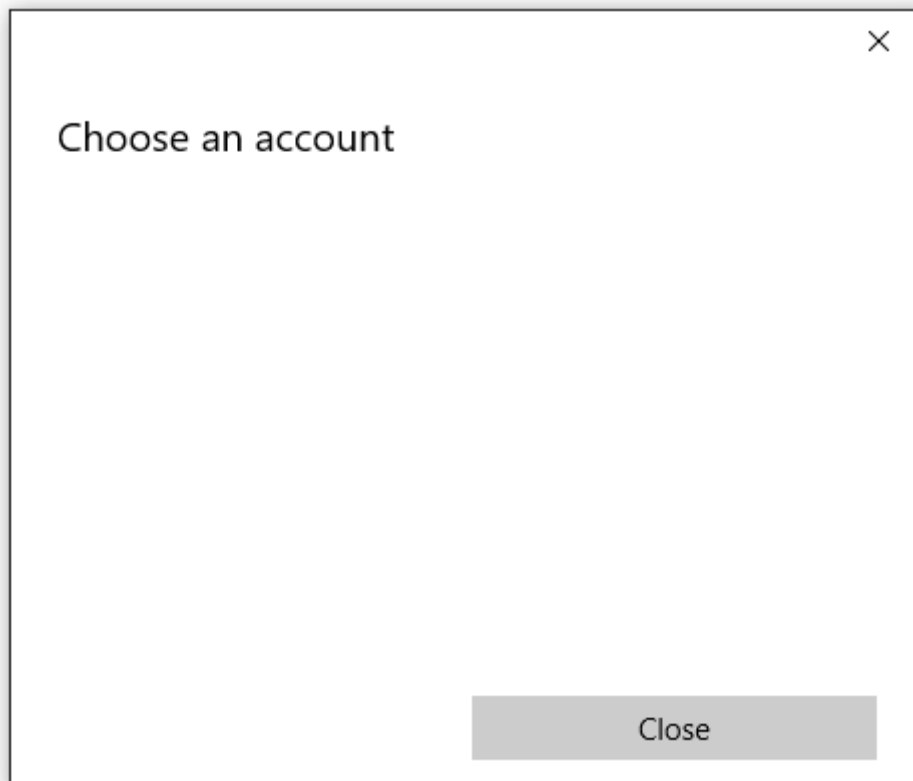
Show the accounts settings pane

The system provides a built-in user interface for managing identity providers and web accounts called **AccountsSettingsPane**. You can show it like this:

C#

```
private void LoginButton_Click(object sender, RoutedEventArgs e)
{
    AccountsSettingsPane.Show();
}
```

If you run your app and click the "Log in" button, it should display an empty window.



The pane is empty because the system only provides a UI shell - it's up to the developer to programmatically populate the pane with the identity providers.

Tip

Optionally, you can use [ShowAddAccountAsync](#) instead of [Show](#), which will return an [IAsyncAction](#), to query for the status of the operation.

Register for AccountCommandsRequested

To add commands to the pane, we start by registering for the `AccountCommandsRequested` event handler. This tells the system to run our build logic when the user asks to see the pane (for example, clicks our XAML button).

In your code behind, override the `OnNavigatedTo` and `OnNavigatedFrom` events and add the following code to them:

C#

```
protected override void OnNavigatedTo(NavigationEventArgs e)
{
    AccountsSettingsPane.GetForCurrentView().AccountCommandsRequested +=
    BuildPaneAsync;
}
```

C#

```
protected override void OnNavigatedFrom(NavigationEventArgs e)
{
    AccountsSettingsPane.GetForCurrentView().AccountCommandsRequested -=
    BuildPaneAsync;
}
```

Users don't interact with accounts very often, so registering and deregistering your event handler in this fashion helps prevent memory leaks. This way, your customized pane is only in memory when there's a high chance a user is going to ask for it (because they're on a "settings" or "login" page, for example).

Build the account settings pane

The `BuildPaneAsync` method is called whenever the **AccountsSettingsPane** is shown. This is where we'll put the code to customize the commands shown in the pane.

Start by obtaining a deferral. This tells the system to delay showing the **AccountsSettingsPane** until we're finished building it.

C#

```
private async void BuildPaneAsync(AccountsSettingsPane s,
    AccountsSettingsPaneCommandsRequestedEventArgs e)
{
    var deferral = e.GetDeferral();

    deferral.Complete();
}
```

Next, get a provider using the `WebAuthenticationCoreManager.FindAccountProviderAsync` method. The URL for the provider varies based on the provider and can be found in the provider's documentation. For Microsoft Accounts and Azure Active Directory, it's "https://login.microsoft.com".

C#

```
private async void BuildPaneAsync(AccountsSettingsPane s,
    AccountsSettingsPaneCommandsRequestedEventArgs e)
{
    var deferral = e.GetDeferral();

    var msaProvider = await
    WebAuthenticationCoreManager.FindAccountProviderAsync(
```

```
        "https://login.microsoft.com", "consumers");

    deferral.Complete();
}
```

Notice that we also pass the string "consumers" to the optional *authority* parameter. This is because Microsoft provides two different types of authentication - Microsoft Accounts (MSA) for "consumers", and Azure Active Directory (AAD) for "organizations". The "consumers" authority indicates we want the MSA option. If you're developing an enterprise app, use the string "organizations" instead.

Finally, add the provider to the **AccountsSettingsPane** by creating a new **WebAccountProviderCommand** like this:

```
C#

private async void BuildPaneAsync(AccountsSettingsPane s,
    AccountsSettingsPaneCommandsRequestedEventArgs e)
{
    var deferral = e.GetDeferral();

    var msaProvider = await
        WebAuthenticationCoreManager.FindAccountProviderAsync(
            "https://login.microsoft.com", "consumers");

    var command = new WebAccountProviderCommand(msaProvider,
        GetMsaTokenAsync);

    e.WebAccountProviderCommands.Add(command);

    deferral.Complete();
}
```

The `GetMsaToken` method we passed to our new **WebAccountProviderCommand** doesn't exist yet (we'll build that in the next step), so feel free to add it as an empty method for now.

Run the above code and your pane should look something like this: